ABSTRACT

AgriTrace is a comprehensive, full-stack web application designed to address the pressing issue of agricultural waste management in India. The platform provides a digital ecosystem connecting farmers, collection agents, and recycling administrators through a role-based system built on modern web technologies.

The system enables farmers to report and list agricultural waste such as wheat stubble, rice residue, corn stover, and other crop residues. Collection agents can browse available waste listings, claim collection tasks, and manage the pickup-to-delivery workflow. Administrators oversee the entire platform, managing users, monitoring analytics, and tracking the recycling pipeline.

Key features include real-time waste tracking with GPS-based location services, photo documentation for waste verification, an integrated payment workflow for agent-to-farmer transactions via Razorpay, a carbon credit tracking system that quantifies environmental impact, and a gamification-based rewards system to incentivize participation. The platform also integrates AI capabilities through Google Genkit for intelligent waste classification.

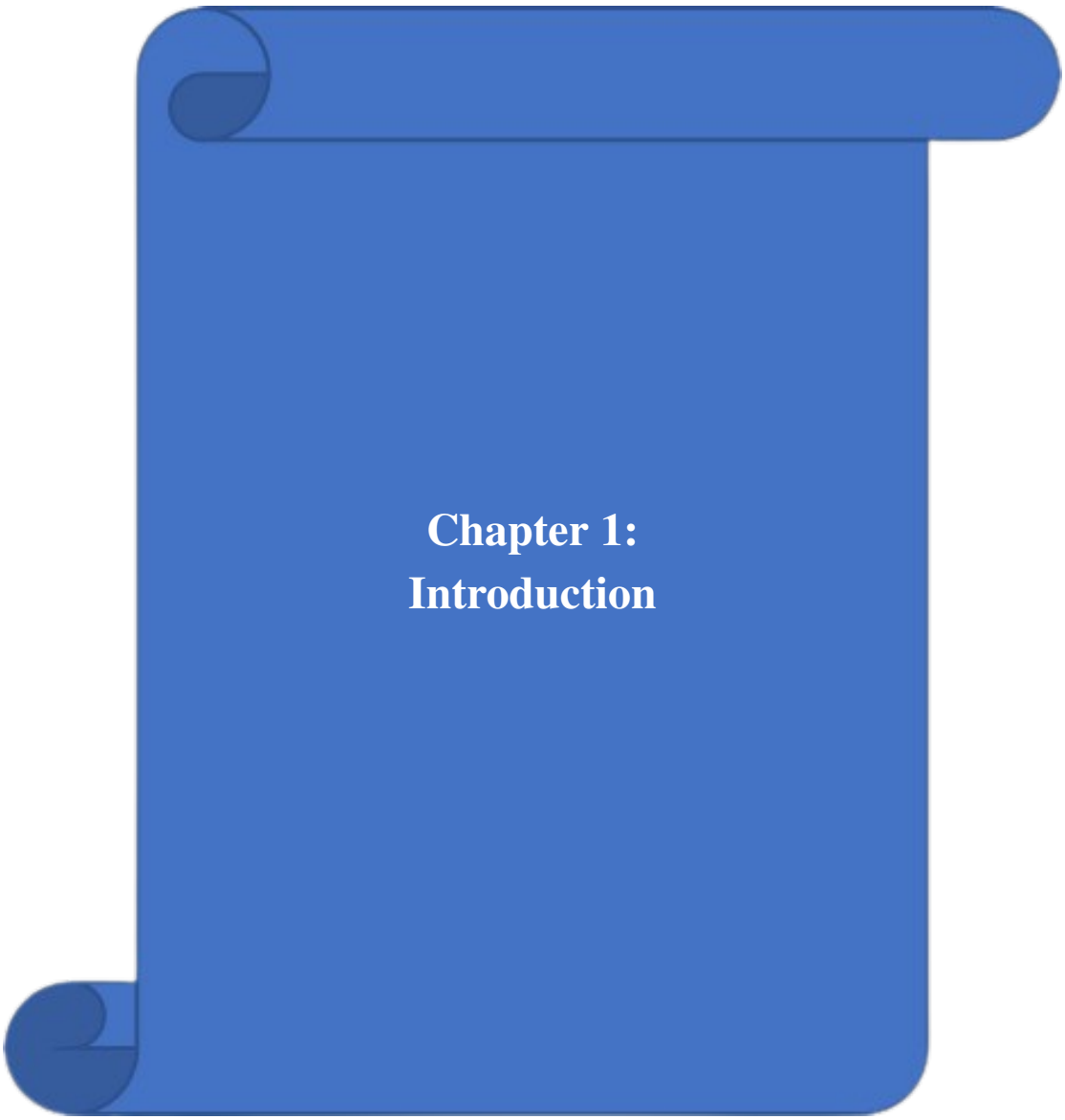
Built with Next.js 15.5, TypeScript, Firebase (Firestore, Authentication, Storage), Tailwind CSS, and Radix UI components, AgriTrace delivers a responsive, modern user experience. The application employs Firestore security rules for data protection and supports real-time data synchronization across all user roles.

AgriTrace contributes to environmental sustainability by diverting agricultural waste from open burning—a major cause of air pollution—toward productive recycling channels, while providing economic incentives to all stakeholders in the waste management chain.

Table of Contents

Figure Index

Table Index


Chapter 1:
Introduction

Agricultural waste management is one of the most critical environmental challenges facing India today. Every year, approximately 500 million tonnes of crop residue is generated, out of which a significant portion is burned in open fields. This practice leads to severe air pollution, soil degradation, loss of nutrients, and contributes significantly to greenhouse gas emissions. The National Capital Region (NCR) witnesses hazardous air quality levels every winter, primarily due to stubble burning in neighboring states.

Despite the environmental and health hazards, farmers often resort to burning because they lack accessible, affordable, and convenient alternatives for waste disposal. There is a disconnect between farmers who generate waste and the recycling industries that can convert this waste into valuable products such as biofuel, compost, animal feed, and building materials.

AgriTrace bridges this gap by providing a digital platform that connects farmers with collection agents and recycling facilities, creating a transparent and efficient agricultural waste management ecosystem. The platform leverages modern web technologies and cloud infrastructure to deliver a real-time, responsive, and scalable solution that can operate across diverse geographical regions.

Aim of the Project

The aim of AgriTrace is to develop a comprehensive web-based platform for tracking, managing, and recycling agricultural waste. The platform facilitates the connection between farmers, waste collection agents, and recycling administrators through a role-based system that ensures efficient workflow management from waste generation to recycling completion.

The specific aims include:

1. Provide farmers with a digital tool to report and list agricultural waste for collection.
2. Enable collection agents to discover, claim, and manage waste pickup tasks efficiently.
3. Empower administrators to oversee the entire waste management pipeline with analytics and user management capabilities.
4. Track the environmental impact through carbon credit calculations.
5. Incentivize participation through a gamification and rewards system.
6. Integrate secure payment processing for waste transactions.

Motivation

The motivation behind AgriTrace stems from several critical factors:

1. **Environmental Crisis:** Open burning of agricultural waste contributes to approximately 25% of air pollution in northern India during October–November. A digital platform can help divert waste from burning to recycling.
2. **Economic Opportunity:** Agricultural waste has significant economic value when processed correctly. Rice husk, wheat stubble, and sugarcane bagasse can be converted into biofuels, compost, and industrial raw materials.
3. **Technology Gap:** While India has made strides in digital agriculture, there is a notable absence of platforms specifically addressing waste logistics, tracking, and recycling coordination.
4. **Policy Support:** The Indian government's push for sustainable agriculture and programs like the National Policy for Management of Crop Residues provide a supportive policy environment for technology-driven solutions.
5. **Carbon Credit Market:** The growing carbon credit ecosystem offers financial incentives for verifiable waste diversion, which AgriTrace can quantify and track.

Technology Overview

AgriTrace is built using a modern, production-grade technology stack carefully selected to deliver reliability, scalability, and excellent developer experience. The following is a brief overview of the core technologies used:

1. **Next.js 15.5:** A React-based framework providing server-side rendering (SSR), static site generation (SSG), API routes, and file-based routing. Next.js enables building full-stack web applications within a single codebase, eliminating the need for a separate backend server. It supports middleware, incremental static regeneration, and optimized image handling out of the box.
2. **TypeScript 5.0:** A statically typed superset of JavaScript that provides compile-time type checking, enhanced IDE support, and improved code maintainability. TypeScript's type system catches errors during development rather than at runtime, significantly reducing bugs in production.
3. **Firebase:** Google's Backend-as-a-Service (BaaS) platform, providing:

   (a) *Cloud Firestore* – A NoSQL document database with real-time synchronization capabilities and offline support.

   (b) *Firebase Authentication* – Managed identity service supporting email/password, OAuth providers, and email verification.

   (c) *Firebase Storage* – Cloud-based file storage for waste report photos and delivery proofs.

4. **Tailwind CSS:** A utility-first CSS framework that enables rapid UI development through composable utility classes. Combined with a custom design system, Tailwind CSS ensures visual consistency across all components.
5. **Radix UI:** An open-source component library providing unstyled, accessible primitives for building high-quality design systems. Radix UI handles complex interactions such as dialogs, dropdowns, and tooltips with proper ARIA attributes.
6. **Razorpay:** India's leading payment gateway, integrated for processing waste transactions. Razorpay supports UPI, credit/debit cards, net banking, and wallet payments with PCI DSS Level 1 compliance.
7. **Google Genkit AI:** A framework for building AI-powered features. AgriTrace uses Genkit for intelligent waste classification, helping farmers identify waste types through AI-assisted categorization.
8. **Cloudinary:** A cloud-based media management service used for uploading, storing, and optimizing waste report photographs with automatic image compression and format conversion.

Chapter 2:
Problem Statement

India generates approximately 500 million tonnes of agricultural waste annually. A substantial portion of this waste, particularly in states like Punjab, Haryana, and Uttar Pradesh, is burned in open fields due to the lack of economically viable and logistically feasible alternatives. This practice results in:

1. **Air Pollution:** Stubble burning contributes to severe air quality deterioration, with PM 2.5 levels reaching hazardous levels in the Indo-Gangetic plains.
2. **Health Hazards:** Respiratory diseases, cardiovascular issues, and allergic reactions increase significantly during the burning season.
3. **Soil Degradation:** Burning destroys beneficial soil microorganisms, reduces organic carbon content, and diminishes soil fertility over time.
4. **Greenhouse Gas Emissions:** Agricultural burning releases CO_2, CO, CH_4, and N_2O, contributing to climate change.

State-wise Crop Residue Generation

The following table presents state-wise crop residue generation and estimated residue burned, based on data from the Indian Agricultural Research Institute (IARI) and the Ministry of Agriculture:

Table 1.1: State-wise Crop Residue Generation and Burning Estimates

State	Residue Generated (MT)	Residue Burned (MT)	% Burned
Punjab	51.0	19.8	38.8
Haryana	27.6	9.1	33.0
Uttar Pradesh	59.0	21.3	36.1
Maharashtra	46.5	7.4	15.9
Madhya Pradesh	33.2	4.3	12.9
Karnataka	28.4	3.8	13.4
Andhra Pradesh	25.1	2.5	10.0
Tamil Nadu	19.8	3.9	19.7
Others	209.4	20.0	9.5
Total	**500.0**	**92.1**	**18.4**

Limitations of Existing Solutions

Several initiatives have been undertaken to address agricultural waste management, but they face significant limitations:

1. **Government Subsidy Programs:** While the government provides subsidies for farm machinery (e.g., Happy Seeder, Super Straw Management System), the adoption rate remains low due to high upfront costs, limited availability, and lack of awareness. Additionally, these programs focus on in-situ management rather than creating a market for waste.
2. **Manual Collection Systems:** Traditional manual collection is labor-intensive, time-consuming, and lacks transparency. There is no mechanism for tracking waste from the point of collection to the recycling facility, leading to inefficiencies and potential mismanagement.
3. **Biomass Power Plants:** Although biomass power plants can utilize agricultural waste, farmers face challenges in transporting waste to these facilities. The absence of a digital platform for coordinating logistics results in poor utilization of available capacity.
4. **Existing Agricultural Apps:** Current agricultural apps (such as Kisan Suvidha, mKisan, etc.) focus primarily on crop management, weather forecasting, and market prices. None of them provide an integrated solution for waste tracking, collection coordination, and environmental impact measurement.

The core problem is the absence of an integrated digital platform that can:

1. Enable farmers to easily report and offer their waste for collection instead of burning it.
2. Provide a transparent marketplace connecting waste generators with waste processors.
3. Track the complete lifecycle of agricultural waste from field to recycling facility.
4. Quantify the environmental impact of waste diversion in terms of carbon credits.
5. Ensure secure and transparent financial transactions for all stakeholders.
6. Provide real-time analytics and oversight for organizational administrators.

AgriTrace addresses each of these gaps by providing a unified, role-based platform with real-time tracking, integrated payments, carbon credit quantification, and comprehensive analytics.

AgriTrace addresses this problem by creating a role-based web platform that digitizes and streamlines the entire agricultural waste management pipeline.

Chapter 3:
Literature Survey

Literature Survey

Agricultural waste management has been extensively studied, but the integration of digital platforms remains relatively nascent. Several papers have explored the environmental impact of crop residue burning, noting that it contributes to nearly 25% of air pollution in Northern India during winter months. Previous attempts to manage this waste focused primarily on mechanical solutions (like Happy Seeders) or localized composting, which suffer from high capital costs and logistical challenges.

Digital platforms for waste management have been successfully deployed in urban solid waste management, but applying these to rural agriculture presents unique challenges. Research indicates that mobile-first architectures, offline capabilities, and vernacular language support are critical for adoption. Furthermore, the integration of real-time GPS tracking and AI-based image verification (e.g., using Google Genkit) represents a novel approach to ensuring transparency and accountability in the waste supply chain.

Our survey concludes that while individual technologies (IoT tracking, cloud databases, payment gateways) exist, a unified platform connecting farmers directly with recycling administrators under a Gamified and incentivized carbon-credit framework is highly necessary.

Chapter 4:
Objectives and Scope of the Project

Project Objectives

The primary objectives of the AgriTrace platform are:

1. **Develop a Role-Based Platform:** Create a web application with distinct interfaces and workflows for Farmers, Collection Agents, and Administrators.
2. **Implement Waste Tracking:** Enable end-to-end tracking of agricultural waste from reporting through collection, transit, and recycling.
3. **Build a Marketplace:** Provide a listing system where farmers can offer waste and agents can claim collection tasks.
4. **Integrate Payment Processing:** Implement Razorpay-based payment processing for waste transactions.
5. **Carbon Credit Tracking:** Calculate and track carbon offsets generated by waste diversion using scientifically established emission factors.
6. **Analytics Dashboard:** Provide comprehensive analytics including waste-by-type distribution, collection-over-time trends, and platform statistics.
7. **Location-Based Services:** Implement GPS-based location tracking for waste reports and collection coordination.
8. **AI Integration:** Leverage Google Genkit AI for intelligent waste classification and recommendations.
9. **Gamification:** Implement a rewards system with points, milestones, and streaks to incentivize platform participation.

Scope of the Project

The scope of AgriTrace encompasses:

1. **User Management:** Registration, authentication, role selection, profile management, and password reset functionality.
2. **Farmer Module:** Waste reporting with photo uploads, listing creation, order tracking, carbon credit viewing, and payment receipt.
3. **Agent Module:** Task discovery via nearby-tasks feature, listing claiming, pickup-to-delivery workflow management, delivery proof uploads, and earnings tracking.

4. **Admin Module:** User management (edit roles, view all users), platform analytics, listing oversight, recycling status management, and statistics dashboard.
5. **Notification System:** Real-time in-app notifications for status changes, new assignments, and system updates.
6. **Responsive Design:** Full support for desktop, tablet, and mobile devices.

Limitations:

1. The platform currently does not support multi-language interfaces.
2. Physical verification of waste quality is outside the scope of the digital platform.
3. The carbon credit values are estimates based on published emission factors and are not certified by any regulatory body.

Chapter 5:
Requirement Analysis

Hardware Requirements

Table 3.1: Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above / AMD Ryzen 3 or above
RAM	4 GB minimum (8 GB recommended)
Storage	256 GB SSD (minimum)
Display	1366 × 768 resolution (minimum)
Network	Broadband internet connection
Client Device	Any device with a modern web browser (Desktop, Tablet, Smartphone)

Software Requirements

Table 3.2: Software Requirements

Component	Specification
Operating System	Windows 10+, macOS 12+, or Linux (Ubuntu 20.04+)
Runtime	Node.js 18+ with npm
Framework	Next.js 15.5 with App Router
Language	TypeScript 5.0
Database	Firebase Firestore (NoSQL, Cloud-hosted)
Authentication	Firebase Authentication
File Storage	Firebase Storage
Payment Gateway	Razorpay API
AI Services	Google Genkit AI
Styling	Tailwind CSS 3.0, Radix UI
Build Tool	Turbopack
Browser	Google Chrome 90+, Firefox 88+, Safari 14+, or Edge 90+
Version Control	Git with GitHub
Deployment	Firebase Hosting / Vercel

Functional Requirements

The following table lists the functional requirements of the AgriTrace platform:

Table 3.3: Functional Requirements

ID	Requirement	Description	Priority
FR-01	User Registration	System shall allow new users to register with email and password, selecting their role (Farmer, Agent, Admin)	High
FR-02	User Authentication	System shall authenticate users via Firebase Authentication with email/password credentials	High
FR-03	Password Reset	System shall allow users to reset their password via email verification link	Medium
FR-04	Waste Listing Creation	Farmers shall be able to create waste listings with title, category, quantity, price, location, photos, and description	High
FR-05	Listing Discovery	Agents shall be able to browse all open waste listings sorted by creation date	High
FR-06	Listing Claiming	Agents shall be able to claim open listings, changing status to ASSIGNED	High
FR-07	Pickup Workflow	Agents shall transition listings through ASSIGNED $\rightarrow$ IN_TRANSIT $\rightarrow$ DELIVERED states	High
FR-08	Listing Cancellation	Farmers shall be able to cancel their own OPEN listings	Medium
FR-09	Payment Processing	System shall integrate Razorpay for processing payments between buyers and sellers	High
FR-10	Carbon Credit Calculation	System shall calculate carbon offsets based on waste type and quantity using established emission factors	Medium
FR-11	Rewards System	System shall track user points, streaks, and milestones based on platform activity	Low

ID	Requirement	Description	Priority
FR-12	Photo Upload	Users shall be able to upload waste photos via Cloudinary integration during listing creation	Medium
FR-13	Location Services	System shall capture GPS coordinates and display map-based location picker for waste reports	Medium
FR-14	Notifications	System shall send in-app notifications for status changes, assignments, and system events	Medium
FR-15	Admin Dashboard	Administrators shall view platform-wide statistics including user counts, listing counts by status, and activity trends	High
FR-16	CSV Export	Farmers shall be able to export their listing data as CSV files for offline record-keeping	Low
FR-17	User Role Management	Administrators shall be able to view all users and edit user roles	High
FR-18	Real-time Updates	All data changes shall be reflected in real-time across all connected clients using Firestore onSnapshot listeners	High

Non-Functional Requirements

The following table lists the non-functional requirements:

Table 3.4: Non-Functional Requirements

ID	Requirement	Description	Priority
NF-01	Performance	Pages shall load within 3 seconds on a standard broadband connection	High
NF-02	Scalability	System shall support up to 10,000 concurrent users without degradation	Medium
NF-03	Security	All data transmission shall be encrypted using HTTPS/TLS. Firebase security rules shall enforce role-based access control	High

ID	Requirement	Description	Priority
NF-04	Availability	System shall maintain 99.5% uptime, leveraging Firebase's SLA guarantees	High
NF-05	Responsiveness	UI shall be fully responsive across desktop (1920×1080), tablet (768×1024), and mobile (375×667) viewports	High
NF-06	Usability	Interface shall follow consistent design patterns with clear navigation, feedback messages, and loading indicators	Medium
NF-07	Maintainability	Codebase shall use TypeScript strict mode, consistent naming conventions, and modular file organization	Medium
NF-08	Data Integrity	Firestore transactions shall ensure atomicity for multi-document operations (e.g., order creation)	High
NF-09	Browser Compatibility	Application shall function correctly on Chrome 90+, Firefox 88+, Safari 14+, and Edge 90+	Medium
NF-10	Accessibility	All interactive elements shall have proper ARIA labels and keyboard navigation support via Radix UI primitives	Low

Chapter 6:
Methodology

The methodology is presented in the same flow as the synopsis: system architecture, modules of the system, and the algorithm.

System Architecture

AgriTrace follows a three-tier architecture to maintain separation of concerns and improve maintainability.

1. **Presentation Layer:** Next.js pages and reusable React components provide role-based interfaces for Farmer, Agent, and Admin users.
2. **Application Layer:** API routes and service modules implement authentication checks, workflow transitions, notifications, and payment processing.
3. **Data Layer:** Firebase Firestore stores transactional data, Firebase Authentication manages identities, and Firebase Storage/Cloudinary handle media assets.

Modules of the System

The AgriTrace system is composed of several tightly integrated modules that handle the end-to-end lifecycle of agricultural waste management. The key modules include:

1. **Authentication and Role Management:** Handles secure login and registration for Farmers, Agents, and Administrators, utilizing Firebase Authentication. It securely routes users to their respective dashboards based on their roles.
2. **Farmer Module (Waste Reporting):** Enables farmers to list their agricultural waste by category, quantity, and price. It includes a geolocation service to tag the exact coordinates of the farm and an image upload utility for visual verification.
3. **Agent Module (Collection Logistics):** Provides collection agents with a map-based or list-based view of available waste tasks. Agents can claim tasks, navigate to the farm using optimized routes, and update the status of the waste to ”In Transit”.
4. **Admin and Analytics Module:** Gives administrators a global view of the system. It tracks total waste diverted, carbon credits generated, and manages users. It utilizes Recharts for real-time visual analytics.

5. **Payment and Incentives Module:** Integrates Razorpay to facilitate secure, instant payments from recyclers to farmers upon successful delivery of waste. It also calculates carbon credits awarded per transaction.

Waste Task Allocation Algorithm

The core algorithm utilized by the platform for matching collection agents with nearby farmers is based on a localized greedy-matching heuristic combined with Haversine distance calculations.

```
def allocate_tasks(agent_location, available_tasks, max_radius_km
    ):
    eligible_tasks = []

    # Filter and calculate distance for all OPEN tasks
    for task in available_tasks:
        if task.status == 'OPEN':
            dist = haversine(agent_location, task.location)
            if dist <= max_radius_km:
                eligible_tasks.append({
                    'task': task,
                    'distance': dist,
                    'priority_score': calculate_priority(task.
                        quantity, dist)
                })

    # Sort by highest priority (combination of proximity and
        waste volume)
    eligible_tasks.sort(key=lambda x: x['priority_score'],
        reverse=True)

    return eligible_tasks
```

Listing 4.1: Agent-Task Matching Algorithm

Development Tools and Environment

The following tools were used throughout the development process:

Table 4.1: Development Tools

Tool	Version	Purpose
Visual Studio Code	1.85+	Primary code editor with TypeScript IntelliSense
Node.js	18.x LTS	JavaScript runtime environment
npm	9.x	Package manager for dependencies
Git	2.40+	Distributed version control
GitHub	–	Repository hosting and collaboration
Firebase CLI	13.x	Firebase project management and deployment
ESLint	8.x	Static code analysis and linting
Prettier	3.x	Code formatting and style enforcement
Chrome DevTools	–	Browser debugging and performance profiling

Razorpay Integration Methodology

The payment processing architecture of AgriTrace relies on a robust integration with the Razorpay API to ensure seamless, secure, and instant financial transactions between waste purchasers (recyclers) and the farmers.

1. **Order Creation:** When a collection agent successfully delivers the waste and initiates payment, the Next.js backend generates a unique Razorpay Order ID securely using server-side API routes.
2. **Client-Side Checkout:** The Order ID is passed to the frontend component which invokes the Razorpay checkout modal. The modal handles the sensitive payment details (cards, UPI, net banking) completely isolated from the AgriTrace servers, maintaining PCI-DSS compliance.
3. **Webhook Verification:** Upon successful payment, Razorpay triggers a secure webhook to the AgriTrace backend. The system mathematically verifies the ‘razorpay_signature‘ using the HMAC SHA256 algorithm and the merchant secret key to prevent spoofing.

4. **State Update:** Only after signature verification is the waste listing marked as ”PAID” and ”DELIVERED” in the Firestore database, executing an atomic transaction to ensure data integrity.

Chapter 7:
System Design and Modeling

Gantt Chart

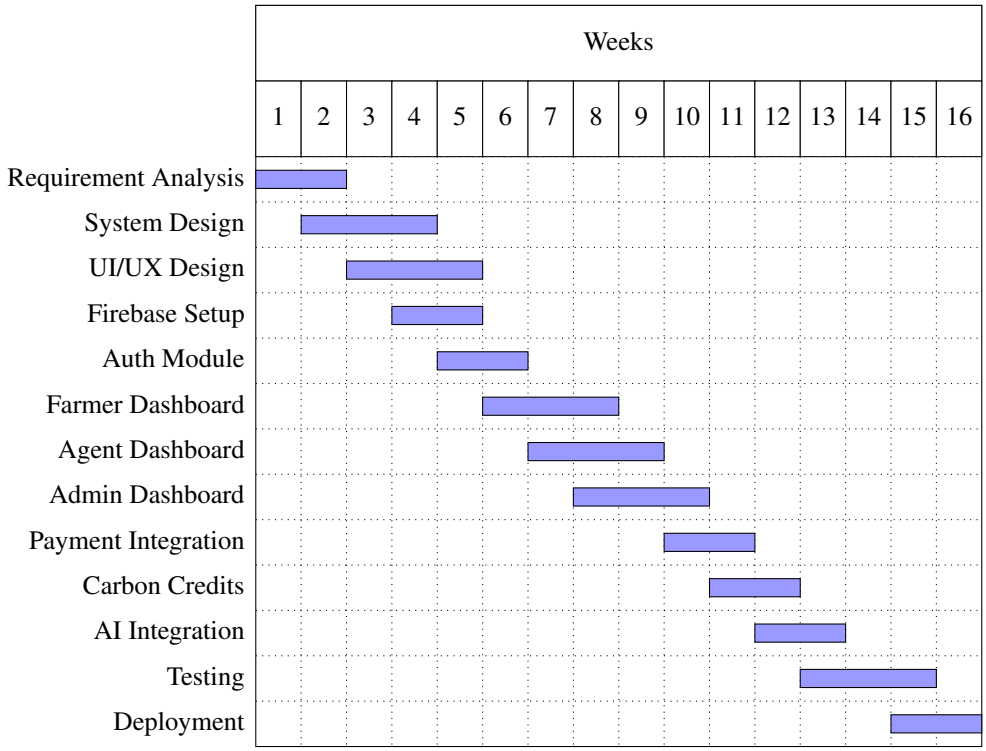


Figure 5.1: Project Development Gantt Chart

DFD (Data Flow Diagram)

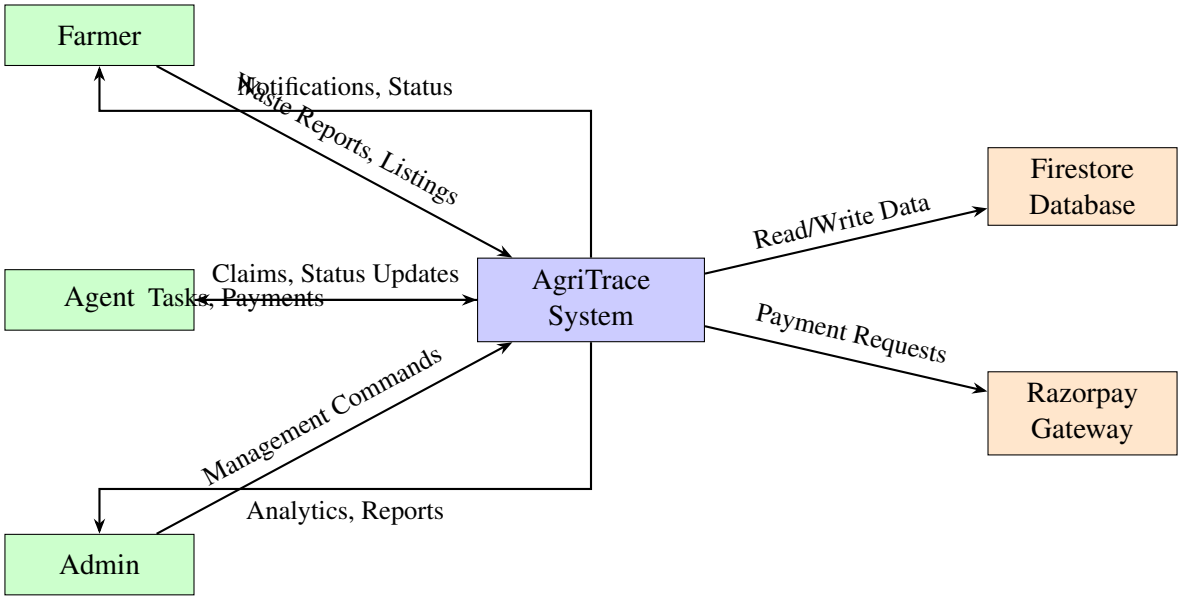


Figure 5.2: Level 0 Data Flow Diagram

Flow Diagram

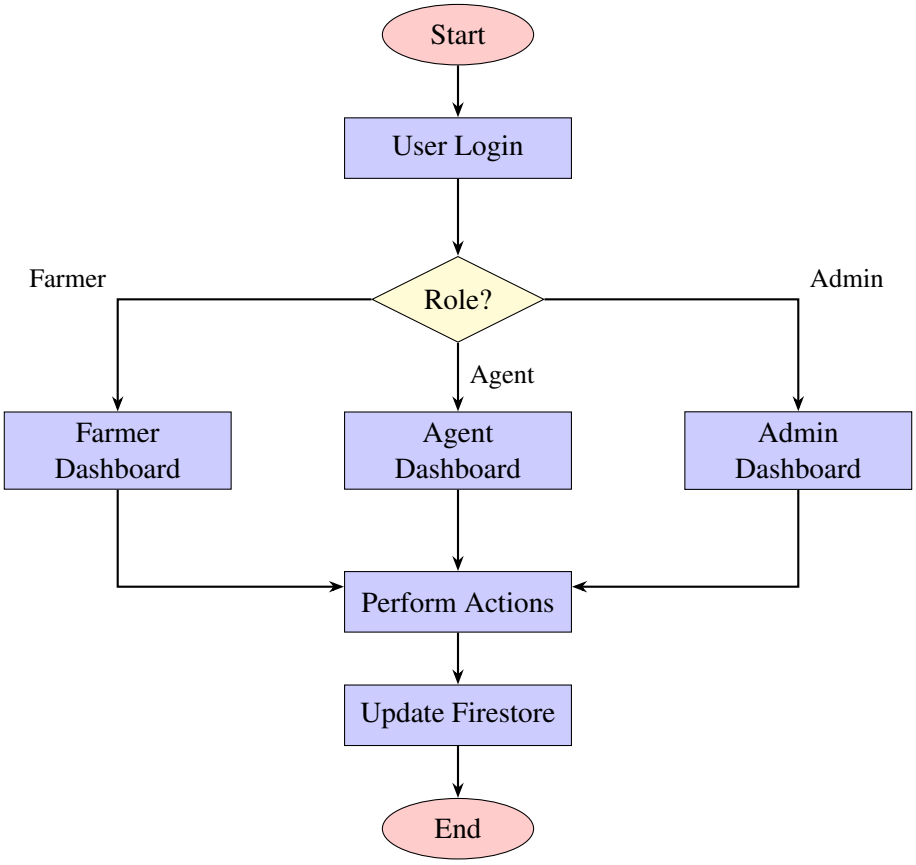


Figure 5.3: Application Flow Diagram

Use Case Diagram

Activity Diagram

ER Diagram

The ER diagram illustrates the six core entities of the AgriTrace system and their relationships. The User entity is central, connecting to Listings (creation relationship), CarbonCredits (earning relationship), and Notifications (receiving relationship). Listings generate Orders, which in turn require Payments. These relationships are implemented as Firestore collections with document references.

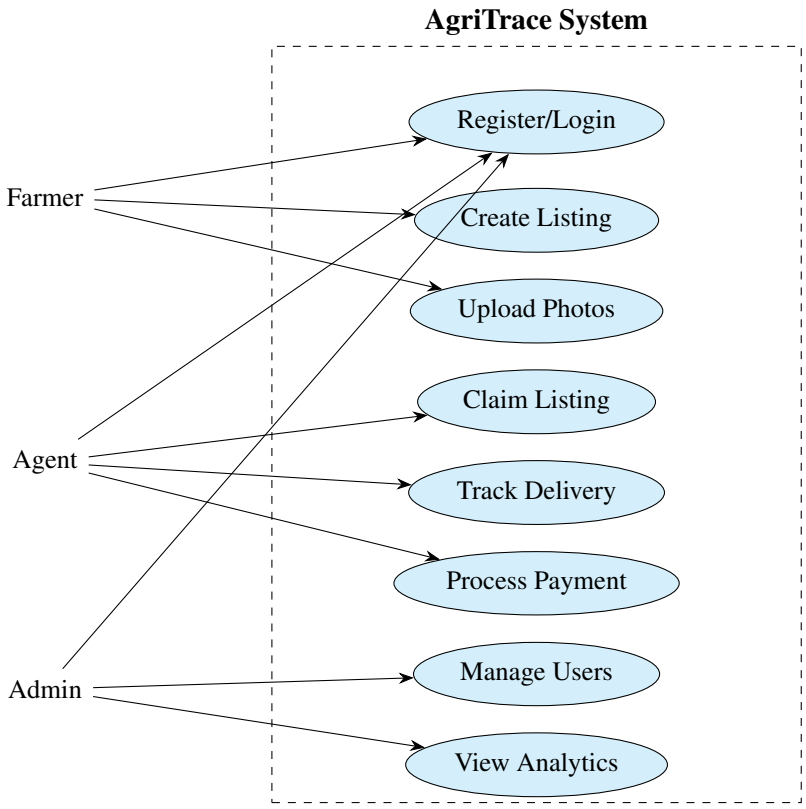


Figure 5.4: Use Case Diagram

Firestore Collection Schema

The following table details the Firestore collections, their fields, data types, and descriptions:

Table 5.1: Firestore Collection Schema – Users

Collection	Field	Type	Description
users	uid	string	Firebase Auth UID (document ID)
users	email	string	User's email address
users	name	string	Display name
users	role	string	User role: farmer, agent, or admin
users	phone	string	Contact phone number
users	verified	boolean	Email verification status
users	profilePhoto	string	URL to profile image
users	location	map	Latitude, longitude, address
users	createdAt	timestamp	Account creation timestamp

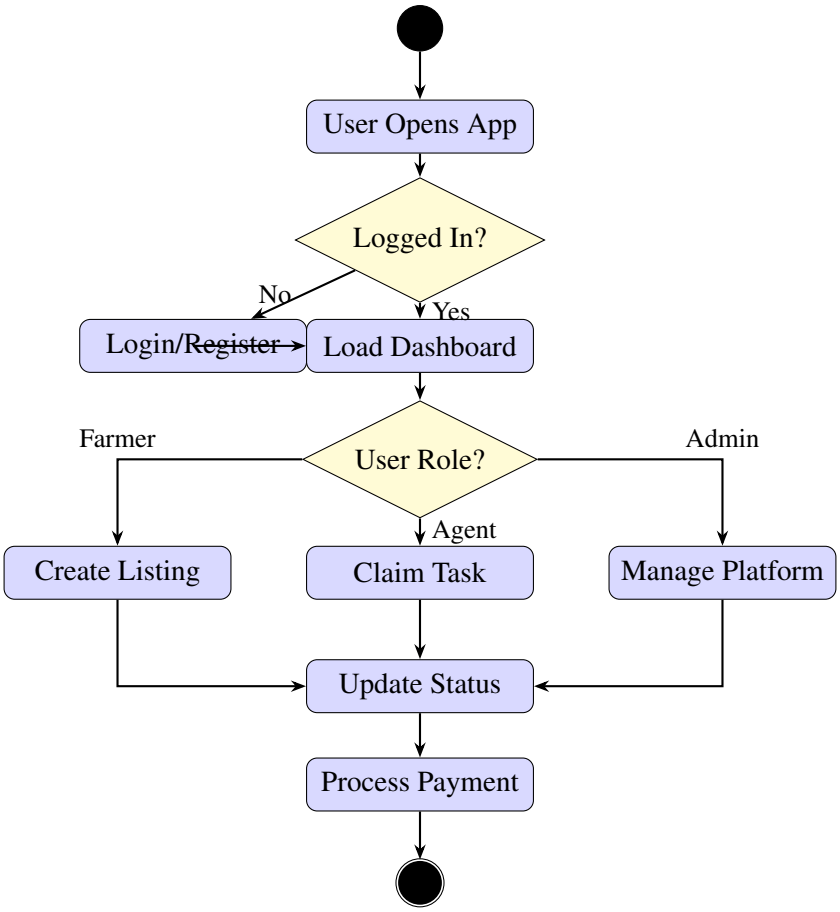


Figure 5.5: Activity Diagram

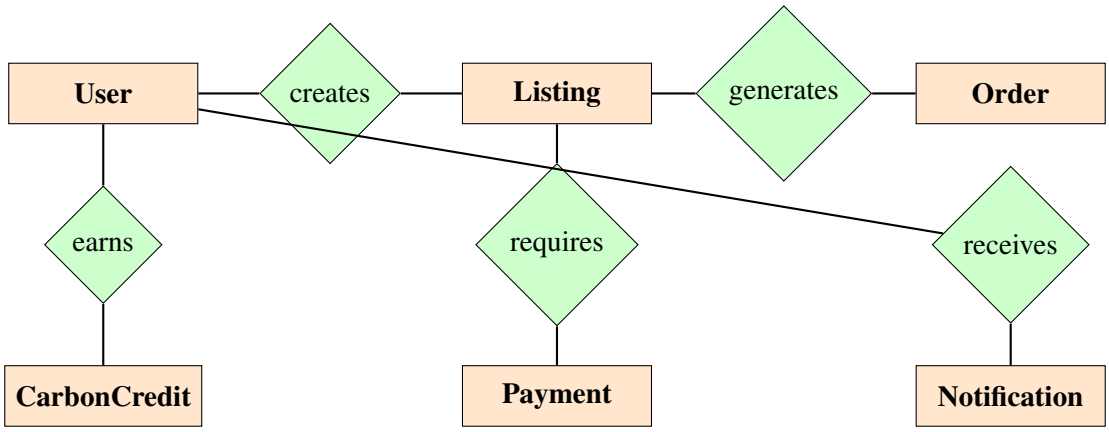


Figure 5.6: Entity-Relationship Diagram

Table 5.2: Firestore Collection Schema – Listings

Collection	Field	Type	Description
listings	id	string	Auto-generated document ID
listings	title	string	Listing title/description

Collection	Field	Type	Description
listings	category	string	Waste type (Rice Husk, Wheat Stubble, etc.)
listings	quantity	number	Quantity in metric tonnes
listings	price	number	Price per unit in INR
listings	description	string	Detailed listing description
listings	photos	array	Array of Cloudinary image URLs
listings	sellerId	string	Reference to seller's user document
listings	sellerEmail	string	Seller's email for quick lookup
listings	status	string	OPEN, ASSIGNED, IN_TRANSIT, DELIVERED, RECYCLED, CANCELLED
listings	assignedAgentId	string	Reference to assigned agent
listings	assignedAgentEmail	string	Agent's email for display
listings	location	map	GPS coordinates and address
listings	assignedAgentEmail	string	Email of the assigned collection agent
listings	paymentStatus	string	Status of payment to farmer (PENDING, PAID)
listings	orderId	string	Reference to the recorded order document
listings	createdAt	timestamp	Listing creation time
listings	updatedAt	timestamp	Last status update time

Table 5.3: Firestore Collection Schema – Orders and Notifications

Collection	Field	Type	Description
orders	id	string	Auto-generated document ID
orders	listingId	string	Reference to listing document

Collection	Field	Type	Description
orders	buyerId	string	Reference to buyer's user document
orders	sellerId	string	Reference to seller's user document
orders	price	number	Transaction amount in INR
orders	status	string	Pending, Accepted, Rejected, Completed
orders	razorpayOrderId	string	Razorpay payment order reference
orders	createdAt	timestamp	Order creation time
notifications	id	string	Auto-generated document ID
notifications	userId	string	Target user for this notification
notifications	title	string	Notification heading
notifications	message	string	Notification body text
notifications	type	string	info, success, warning, error
notifications	read	boolean	Whether user has read the notification
notifications	link	string	Optional deep link to related content
notifications	createdAt	timestamp	Notification creation time

Component Hierarchy

The AgriTrace application follows a hierarchical component structure aligned with the Next.js App Router pattern. The following describes the component tree:

1. **Root Layout** (`app/layout.tsx`)
   (a) `AuthProvider` – Wraps the entire app with authentication context
   (b) `Toaster` – Global toast notification component
   (c) **Landing Page** (`app/page.tsx`) – Public-facing homepage
   (d) **Login Page** (`app/login/page.tsx`)
      i. `LoginForm` – Email/password input with validation
      ii. `TestimonialCarousel` – Rotating user testimonials
   (e) **Signup Page** (`app/signup/page.tsx`)
      i. `SignupForm` – Registration with role selection
   (f) **Dashboard Page** (`app/dashboard/page.tsx`)
      i. `FarmerDashboard` – Farmers-only content
         A. `StatCard` – Statistics display cards
         B. `PhotoUploader` – Image upload component
         C. `LocationPicker` – GPS coordinate capture
         D. `PaymentButton` – Razorpay integration
         E. `ListingCard` – Individual listing display
      ii. `AgentDashboard` – Agents-only content
         A. `StatCard` – Statistics display cards
         B. `TaskCard` – Available/claimed task cards
   (g) **Admin Page** (`app/admin/page.tsx`)
      i. `AdminStatCards` – Platform-wide statistics
      ii. `UserManagement` – User list with role editing
      iii. `ListingManagement` – All listings with recycling controls
      iv. `AnalyticsCharts` – Recharts-based data visualizations
   (h) **Shared Components** (`components/ui/`)

i. `Button`, `Dialog`, `Card`, `Badge` – Radix UI primitives

ii. `Select`, `Input`, `Label` – Form elements

iii. `Toast`, `Tabs`, `Sheet` – Interactive elements

Chapter 8:
Coding

This chapter presents the key code modules of the AgriTrace platform. The codebase follows a modular architecture with clear separation of concerns.

Firebase Configuration

```
import { initializeApp, getApps, getApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import { getStorage } from 'firebase/storage';

const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.
     NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
};

const app = !getApps().length ? initializeApp(firebaseConfig) :
   getApp();
export const auth = getAuth(app);
export const db = getFirestore(app);
export const storage = getStorage(app);
```

Listing 6.1: Firebase Initialization (firebase.ts)

Firebase Service Layer

```
// Create a new waste listing
export async function createListing(
  data: Omit<Listing, 'id' | 'createdAt' | 'updatedAt' | 'status
     '>
): Promise<string> {
  const docRef = await addDoc(collection(db, 'listings'), {
    ...data,
    status: ListingStatus.OPEN,
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  });
  return docRef.id;
}
```

```
13
14 // Agent claims a listing
15 export async function claimListing(
16   listingId: string, agentId: string, agentEmail: string
17 ): Promise<void> {
18   await updateDoc(doc(db, 'listings', listingId), {
19     status: ListingStatus.ASSIGNED,
20     assignedAgentId: agentId,
21     assignedAgentEmail: agentEmail,
22     updatedAt: serverTimestamp(),
23   });
24 }
```

Listing 6.2: Core Service Functions (firebase-service.ts)

Authentication Context

```
1  // AuthContext provides authentication state across the app
2  export function AuthProvider({ children }: { children: ReactNode
       }) {
3    const [user, setUser] = useState<User | null>(null);
4    const [loading, setLoading] = useState(true);
5
6    useEffect(() => {
7      const unsubscribe = onAuthStateChanged(auth, async (
           firebaseUser) => {
8        if (firebaseUser) {
9          const userDoc = await getDoc(doc(db, 'users',
               firebaseUser.uid));
10         setUser({ ...firebaseUser, role: userDoc.data()?.role });
11       } else {
12         setUser(null);
13       }
14       setLoading(false);
15     });
16     return () => unsubscribe();
17   }, []);
18
19   return (
20     <AuthContext.Provider value={{ user, loading }}>
21       {children}
22     </AuthContext.Provider>
23   );
```

```
}
```

Listing 6.3: Auth Context Provider

Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAuthenticated() {
      return request.auth != null;
    }

    match /users/{userId} {
      allow read: if isAuthenticated();
      allow create: if isAuthenticated()
                    && request.auth.uid == userId;
      allow update: if isAuthenticated();
      allow delete: if false;
    }

    match /listings/{listingId} {
      allow read: if isAuthenticated();
      allow create: if isAuthenticated();
      allow update: if isAuthenticated();
      allow delete: if isAuthenticated()
                    && request.auth.uid == resource.data.sellerId
                       ;
    }
  }
}
```

Listing 6.4: Firestore Security Rules (firestore.rules)

Carbon Credit Calculation

```
// Carbon factors (kg CO2 saved per MT of waste diverted)
export const CARBON_FACTORS: Record<string, number> = {
  'Rice Husk': 1320,
  'Rice Residue': 1280,
  'Wheat Stubble': 1150,
  'Sugarcane Bagasse': 980,
  'Cotton Stalks': 1050,
```

```
  'Corn Stover': 1100,
  'Paddy Straw': 1300,
  'Coconut Shell': 1400,
  'Groundnut Shell': 1200,
  'Mustard Stalks': 1080,
  'Other': 1000,
};

// Calculate carbon offset for diverted waste
export function calculateCarbonOffset(
  wasteType: string, quantityMT: number
): number {
  const factor = CARBON_FACTORS[wasteType] || CARBON_FACTORS['
     Other'];
  return factor * quantityMT;
}
```

Listing 6.5: Carbon Credit Service (carbon-service.ts)

The `CARBON_FACTORS` constant maps each waste type to its CO_2 equivalent savings in kilograms per metric tonne. The `calculateCarbonOffset` function provides a simple multiplication-based calculation with a fallback to the "Other" category for unrecognized waste types.

TypeScript Type Definitions

AgriTrace uses TypeScript interfaces and enums to enforce type safety across the entire codebase. The `types.ts` module defines all data models used by the application:

```
import { Timestamp } from 'firebase/firestore';

export enum UserRole {
  FARMER = 'farmer',
  AGENT = 'agent',
  ADMIN = 'admin',
}

export enum WasteReportStatus {
  PENDING_PICKUP = 'Pending Pickup',
  IN_TRANSIT = 'In Transit',
  DELIVERED = 'Delivered',
  PROCESSING = 'Processing',
  RECYCLED = 'Recycled',
  REJECTED = 'Rejected',
  COMPLETED = 'Completed',
```

```
}

export interface LocationData {
  latitude: number;
  longitude: number;
  address?: string;
  timestamp?: number;
}

export interface WasteReport {
  id: string;
  farmerName: string;
  farmerId: string;
  wasteType: string;
  quantity: number;
  location: LocationData;
  photos?: string[];
  notes?: string;
  status: WasteReportStatus;
  collectionAgent?: string;
  collectionAgentId?: string;
  reportedAt: Timestamp | Date;
  lastUpdate: Timestamp | Date;
  payment?: number;
  paymentStatus?: 'Pending' | 'Paid';
}

export enum ListingStatus {
  OPEN = 'OPEN',
  ASSIGNED = 'ASSIGNED',
  IN_TRANSIT = 'IN_TRANSIT',
  DELIVERED = 'DELIVERED',
  RECYCLED = 'RECYCLED',
  CANCELLED = 'CANCELLED',
}

export interface Listing {
  id?: string;
  title: string;
  price: number;
  quantity?: number;
  location?: LocationData;
```

```
  category: string;
  sellerId?: string;
  sellerEmail?: string;
  description?: string;
  photos?: string[];
  status?: ListingStatus;
  assignedAgentId?: string;
  assignedAgentEmail?: string;
  createdAt?: Timestamp | Date;
  updatedAt?: Timestamp | Date;
}

export interface UserProfile {
  uid: string;
  email: string;
  role: UserRole;
  name?: string;
  phone?: string;
  location?: LocationData;
  profilePhoto?: string;
  createdAt: Timestamp | Date;
  verified: boolean;
}

export interface CarbonCredit {
  id?: string;
  userId: string;
  userRole: 'farmer' | 'agent' | 'admin';
  listingId: string;
  wasteType: string;
  quantityMT: number;
  carbonOffsetKg: number;
  creditDate: Timestamp | Date;
  status: 'pending' | 'verified' | 'redeemed';
}

export interface Order {
  id?: string;
  listingId: string;
  buyerId: string;
  sellerId: string;
  price: number;
```

```
  status?: 'Pending' | 'Accepted' | 'Rejected' | 'Completed';
  createdAt?: Timestamp | Date;
}
```

Listing 6.6: Type Definitions (types.ts)

The type definitions ensure that all data flowing through the application conforms to a strict schema. The `LocationData` interface standardizes geolocation data with optional address fields. Enums like `ListingStatus` and `WasteReportStatus` enumerate all valid states, preventing invalid status assignments at compile time.

Location Service

The location service module provides GPS-based geolocation functionality and reverse geocoding:

```
export interface Location {
  latitude: number;
  longitude: number;
}

export function getCurrentLocation(): Promise<Location> {
  return new Promise((resolve, reject) => {
    if (!navigator.geolocation) {
      reject(new Error('Geolocation is not supported'));
      return;
    }
    navigator.geolocation.getCurrentPosition(
      (position) => {
        resolve({
          latitude: position.coords.latitude,
          longitude: position.coords.longitude,
        });
      },
      (error) => { reject(error); },
      { enableHighAccuracy: true, timeout: 15000, maximumAge: 0 }
    );
  });
}

export async function reverseGeocode(
  lat: number, lng: number
): Promise<string> {
  try {
```

```
    const response = await fetch(
      `https://nominatim.openstreetmap.org/reverse` +
      `?format=json&lat=${lat}&lon=${lng}`
    );
    const data = await response.json();
    return data.display_name || `${lat}, ${lng}`;
  } catch {
    return `${lat.toFixed(6)}, ${lng.toFixed(6)}`;
  }
}

export function calculateDistance(
  loc1: Location, loc2: Location
): number {
  const R = 6371; // Earth radius in km
  const dLat = ((loc2.latitude - loc1.latitude) * Math.PI) / 180;
  const dLon = ((loc2.longitude - loc1.longitude) * Math.PI) /
     180;
  const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos((loc1.latitude * Math.PI) / 180) *
    Math.cos((loc2.latitude * Math.PI) / 180) *
    Math.sin(dLon / 2) * Math.sin(dLon / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c;
}
```

Listing 6.7: Location Service (location-service.ts)

The location service utilizes the browser's Geolocation API for acquiring GPS coordinates and OpenStreetMap's Nominatim service for reverse geocoding. The `calculateDistance` function implements the Haversine formula to compute the great-circle distance between two geographic points, which is used to sort nearby waste listings for collection agents.

Notification Service

The notification service manages in-app notifications for status updates and system events:

```
import { collection, addDoc, query, where, orderBy,
         getCountFromServer, updateDoc, doc,
         serverTimestamp } from 'firebase/firestore';
import { db } from './firebase';

export interface AppNotification {
  id?: string;
```

```
  userId: string;
  title: string;
  message: string;
  type: 'info' | 'success' | 'warning' | 'error';
  read: boolean;
  createdAt: any;
  link?: string;
}

export async function createNotification(
  data: Omit<AppNotification, 'id' | 'read' | 'createdAt'>
): Promise<string> {
  const ref = await addDoc(collection(db, 'notifications'), {
    ...data,
    read: false,
    createdAt: serverTimestamp(),
  });
  return ref.id;
}

export async function getUnreadCount(
  userId: string
): Promise<number> {
  const q = query(
    collection(db, 'notifications'),
    where('userId', '==', userId),
    where('read', '==', false)
  );
  const snapshot = await getCountFromServer(q);
  return snapshot.data().count;
}

export async function markAsRead(
  notificationId: string
): Promise<void> {
  await updateDoc(doc(db, 'notifications', notificationId), {
    read: true,
  });
}
```

Listing 6.8: Notification Service (notification-service.ts)

Notifications are stored as Firestore documents, enabling real-time synchronization. The `getUnreadCount` function uses Firestore's `getCountFromServer` for efficient counting without downloading entire documents.

Storage Service

The storage service handles file uploads to Cloudinary for waste report photos and delivery proofs:

```
const CLOUD_NAME = process.env.NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME;
const UPLOAD_PRESET = process.env.
   NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET;

export async function uploadImage(
  file: File
): Promise<string> {
  const formData = new FormData();
  formData.append('file', file);
  formData.append('upload_preset', UPLOAD_PRESET || '');
  formData.append('folder', 'agritrace');

  const response = await fetch(
    `https://api.cloudinary.com/v1_1/${CLOUD_NAME}/image/upload`,
    { method: 'POST', body: formData }
  );

  if (!response.ok) {
    throw new Error('Upload failed');
  }
  const data = await response.json();
  return data.secure_url;
}

export async function uploadMultipleImages(
  files: File[]
): Promise<string[]> {
  const uploadPromises = files.map((file) => uploadImage(file));
  return Promise.all(uploadPromises);
}
```

Listing 6.9: Storage Service (storage-service.ts)

Image uploads use Cloudinary's unsigned upload preset mechanism, which avoids exposing API secrets on the client side. The `uploadMultipleImages` function leverages `Promise.all`

for concurrent uploads, minimizing total upload time when farmers submit multiple waste photographs.

Payment Button Component

The `PaymentButton` component integrates Razorpay's checkout flow with a custom React UI:

```
'use client';
import { useState } from 'react';
import { Button } from './ui/button';

interface PaymentButtonProps {
    amount: number;
    listingId: string;
    buyerId: string;
    sellerId: string;
    onSuccess?: (orderId: string) => void;
    children?: React.ReactNode;
    className?: string;
    variant?: 'default' | 'outline' | 'ghost';
}

export default function PaymentButton({
    amount, listingId, buyerId, sellerId,
    onSuccess, children, className, variant = 'default'
}: PaymentButtonProps) {
    const [loading, setLoading] = useState(false);

    const handlePayment = async () => {
        setLoading(true);
        try {
            // Step 1: Create Razorpay order
            const res = await fetch('/api/razorpay/create', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ amount, listingId, buyerId
                    , sellerId }),
            });
            const { razorpayOrderId } = await res.json();

            // Step 2: Initialize Razorpay Checkout
            const options = {
                key: process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID,
```

```
                amount: amount * 100,
                currency: 'INR',
                order_id: razorpayOrderId,
                handler: async function (response: any) {
                    // Verify payment on server
                    const verifyRes = await fetch('/api/razorpay/
                       verify', {
                        method: 'POST',
                        headers: { 'Content-Type': 'application/
                           json' },
                        body: JSON.stringify({
                            razorpay_order_id: response.
                               razorpay_order_id,
                            razorpay_payment_id: response.
                               razorpay_payment_id,
                            razorpay_signature: response.
                               razorpay_signature,
                            listingId, buyerId, sellerId, price:
                               amount,
                        }),
                    });
                    const verifyData = await verifyRes.json();
                    if (verifyData.ok) onSuccess?.(verifyData.
                       orderId);
                }
            };
            const rzp = new (window as any).Razorpay(options);
            rzp.open();
        } catch (err) { console.error('Payment error:', err); }
        finally { setLoading(false); }
    };

    return (
        <Button onClick={handlePayment} disabled={loading}
                classname={className} variant={variant}>
            {children}
        </Button>
    );
}
```

Listing 6.10: Payment Button Component (payment-button.tsx)

The payment flow follows a two-step process: (1) a server-side API call creates a Razorpay order with the amount and metadata, and (2) the Razorpay checkout modal is launched on the client side. This ensures that payment amounts cannot be tampered with by the client.

Razorpay Payment API Routes

The Next.js API handles server-side order creation and cryptographic verification. The order creation route initiates the transaction with Razorpay:

```
import Razorpay from 'razorpay';
import { NextResponse } from 'next/server';

const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID!,
  key_secret: process.env.RAZORPAY_KEY_SECRET!,
});

export async function POST(request: Request) {
  try {
    const { amount } = await request.json();
    const razorpayOrder = await razorpay.orders.create({
      amount: amount * 100, // paise
      currency: 'INR',
      receipt: `receipt_${Date.now()}`,
    });
    return NextResponse.json({ razorpayOrderId: razorpayOrder.id
       });
  } catch (error) {
    return NextResponse.json({ error: 'Failed' }, { status: 500
       });
  }
}
```

Listing 6.11: Razorpay Order Creation (api/razorpay/create/route.ts)

The function returns the Razorpay order ID which is then used by the client-side checkout modal. After the user completes the payment, the following verification route ensures the payment's cryptographic integrity:

```
import { NextResponse } from 'next/server';
import crypto from 'crypto';

export async function POST(req: Request) {
  try {
    const { razorpay_order_id, razorpay_payment_id,
```

```
            razorpay_signature, listingId, buyerId,
            sellerId, price } = await req.json();

    const body = razorpay_order_id + "|" + razorpay_payment_id;
    const expectedSignature = crypto
      .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET!)
      .update(body.toString())
      .digest('hex');

    if (expectedSignature !== razorpay_signature) {
      return NextResponse.json({ error: 'Invalid' }, { status:
        400 });
    }

    // Use Firebase Admin for reliable server-side persistence
    const { db } = await import('@/lib/firebase-server');
    const { FieldValue } = await import('firebase-admin/firestore
      ');

    const orderRef = await db.collection('orders').add({
      listingId, buyerId, sellerId, price,
      status: 'Paid', createdAt: FieldValue.serverTimestamp(),
    });

    await db.collection('listings').doc(listingId).update({
      paymentStatus: 'PAID', orderId: orderRef.id,
      updatedAt: FieldValue.serverTimestamp(),
    });

    return NextResponse.json({ ok: true, orderId: orderRef.id });
  } catch (err) {
    return NextResponse.json({ error: 'Error' }, { status: 500 })
      ;
  }
}
```

Listing 6.12: Razorpay Payment Verification (api/razorpay/verify/route.ts)

This verification step is critical for security as it prevents malicious users from forging payment success responses. By performing this check on the server using the RAZORPAY_KEY_SECRET and then updating the database using the Firebase Admin SDK, we ensure that the transaction is legitimate and the listing status is accurately reflected.

Admin Statistics Service

The admin statistics function aggregates platform-wide metrics by querying multiple Firestore collections:

```
export async function getAdminStats() {
  try {
    const usersRef = collection(db, 'users');
    const farmersQuery = query(usersRef,
      where('role', '==', 'FARMER'));
    const agentsQuery = query(usersRef,
      where('role', '==', 'AGENT'));

    const [farmersSnap, agentsSnap] = await Promise.all([
      getCountFromServer(farmersQuery),
      getCountFromServer(agentsQuery),
    ]);

    const listingsRef = collection(db, 'listings');
    const openQuery = query(listingsRef,
      where('status', '==', 'OPEN'));
    const deliveredQuery = query(listingsRef,
      where('status', '==', 'DELIVERED'));

    const [openSnap, deliveredSnap] = await Promise.all([
      getCountFromServer(openQuery),
      getCountFromServer(deliveredQuery),
    ]);

    return {
      users: {
        farmers: farmersSnap.data().count,
        agents: agentsSnap.data().count,
        total: farmersSnap.data().count +
               agentsSnap.data().count,
      },
      listings: {
        open: openSnap.data().count,
        delivered: deliveredSnap.data().count,
      },
    };
  } catch (error) {
    return {
```

```
      users: { farmers: 0, agents: 0, total: 0 },
      listings: { open: 0, delivered: 0 },
    };
  }
}
```

Listing 6.13: Admin Stats (firebase-service.ts – getAdminStats)

The function uses `getCountFromServer` for efficient aggregation without downloading individual documents, reducing bandwidth consumption. Error handling returns zeroed-out stats to prevent the admin dashboard from crashing if Firestore is temporarily unavailable.

Waste Delivery Tracking

The waste delivery tracking functions manage the logistics of waste transportation from collection agents to recycling facilities:

```
export async function recordWasteDelivery(
  data: any
): Promise<string> {
  const payload = {
    ...data,
    createdAt: serverTimestamp(),
    status: 'DELIVERED',
  };
  const ref = await addDoc(
    collection(db, 'wasteDeliveries'), payload
  );

  if (data.reportId) {
    const reportRef = doc(db, 'wasteReports', data.reportId);
    await updateDoc(reportRef, {
      status: 'IN_TRANSIT',
      collectionAgentId: data.agentId,
      collectionAgent: data.agentName,
      lastUpdate: serverTimestamp(),
    });
  }
  return ref.id;
}

export async function recordWasteProcessing(
  data: any
): Promise<string> {
```

```
  const payload = {
    ...data,
    createdAt: serverTimestamp(),
    status: 'RECYCLED',
  };
  const ref = await addDoc(
    collection(db, 'wasteProcessing'), payload
  );

  if (data.intakeId) {
    const intakeRef = doc(db, 'wasteIntakes', data.intakeId);
    await updateDoc(intakeRef, {
      status: 'PROCESSED',
      lastUpdate: serverTimestamp(),
    });
  }
  return ref.id;
}
```

Listing 6.14: Waste Delivery Functions (firebase-service.ts)

These functions implement a cascading update pattern where recording a delivery or processing event automatically updates the parent document's status. This ensures data consistency across the waste management pipeline without requiring client-side orchestration.

Listing Status Workflow Functions

The following functions implement the complete listing status transition workflow:

```
export async function startPickup(
  listingId: string
): Promise<void> {
  const ref = doc(db, 'listings', listingId);
  await updateDoc(ref, {
    status: 'IN_TRANSIT' as ListingStatus,
    updatedAt: serverTimestamp(),
  });
}

export async function markDelivered(
  listingId: string
): Promise<void> {
  const ref = doc(db, 'listings', listingId);
  await updateDoc(ref, {
```

```
    status: 'DELIVERED' as ListingStatus,
    updatedAt: serverTimestamp(),
  });
}

export async function cancelListing(
  listingId: string
): Promise<void> {
  const ref = doc(db, 'listings', listingId);
  await updateDoc(ref, {
    status: 'CANCELLED' as ListingStatus,
    updatedAt: serverTimestamp(),
  });
}
```

Listing 6.15: Listing Workflow (firebase-service.ts)

Each function targets a specific listing document by ID and atomically updates both the status field and the `updatedAt` timestamp. The status transitions follow a strict sequence: OPEN → ASSIGNED → IN_TRANSIT → DELIVERED → RECYCLED, with CANCELLED available from any active state.

Chapter 9:
Output

This chapter presents the output of the AgriTrace platform, describing the user-facing interfaces and features across all modules. The descriptions are organized by functional area and cover the visual design, interactive elements, and data presentation for each screen.

Project Screenshots

The following screenshots show the key interfaces of the AgriTrace web application, based on the latest provided captures.

Figure 7.1: Landing Page Hero Section

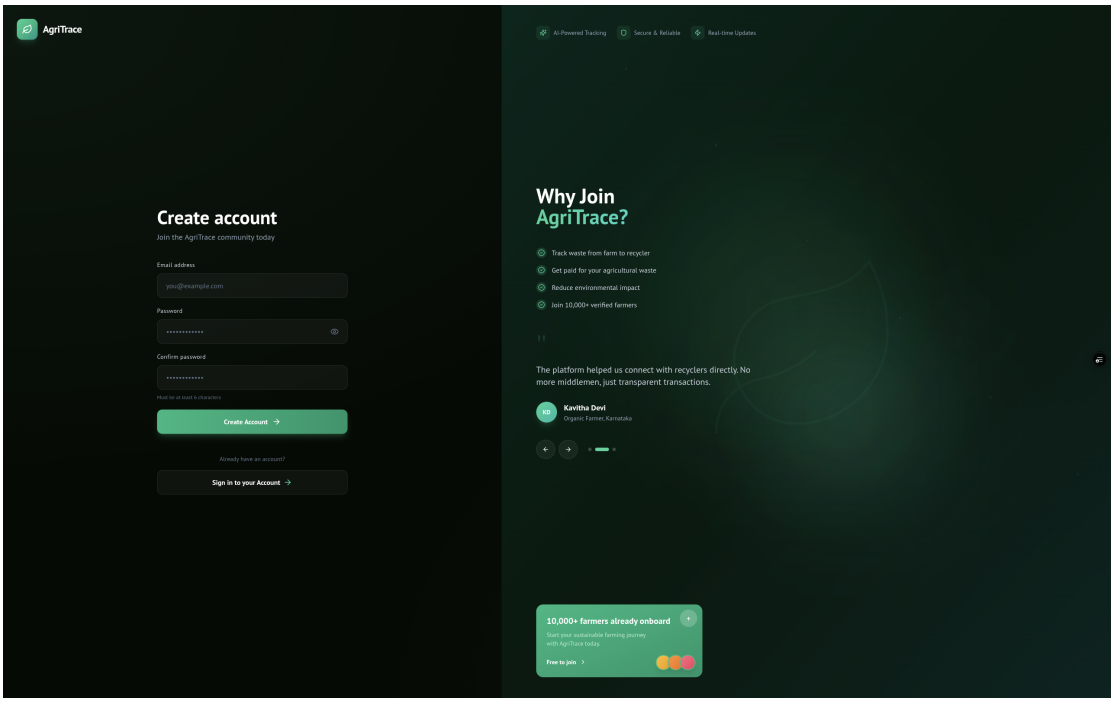


Figure 7.2: Login Page

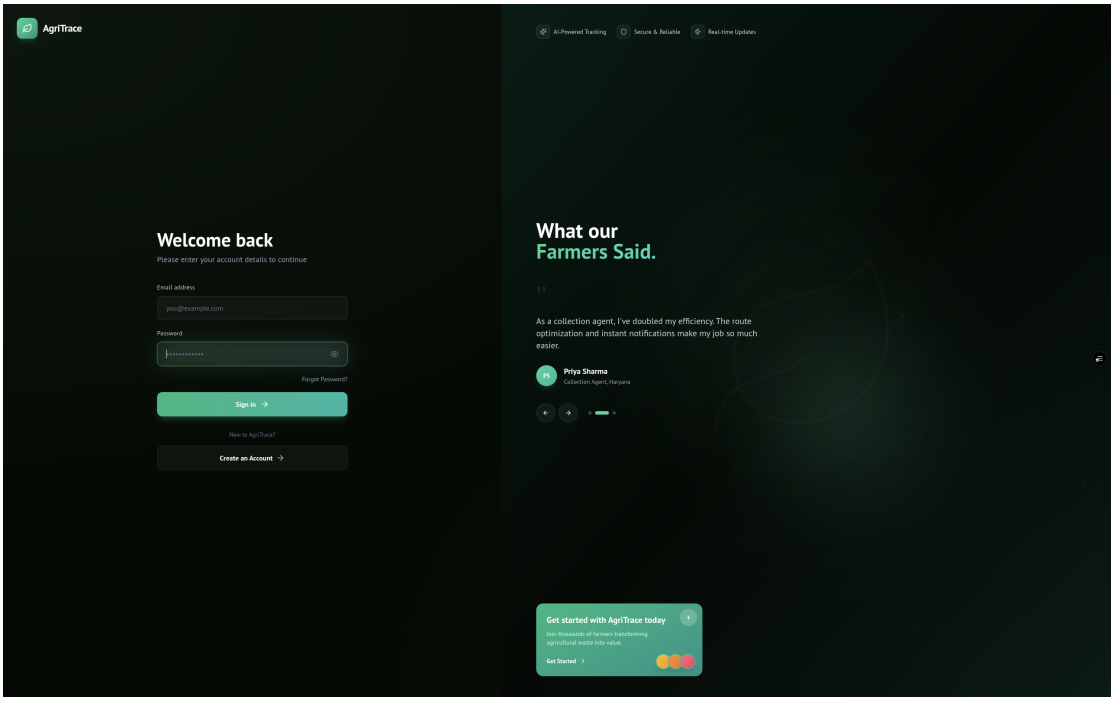


Figure 7.3: Signup Page

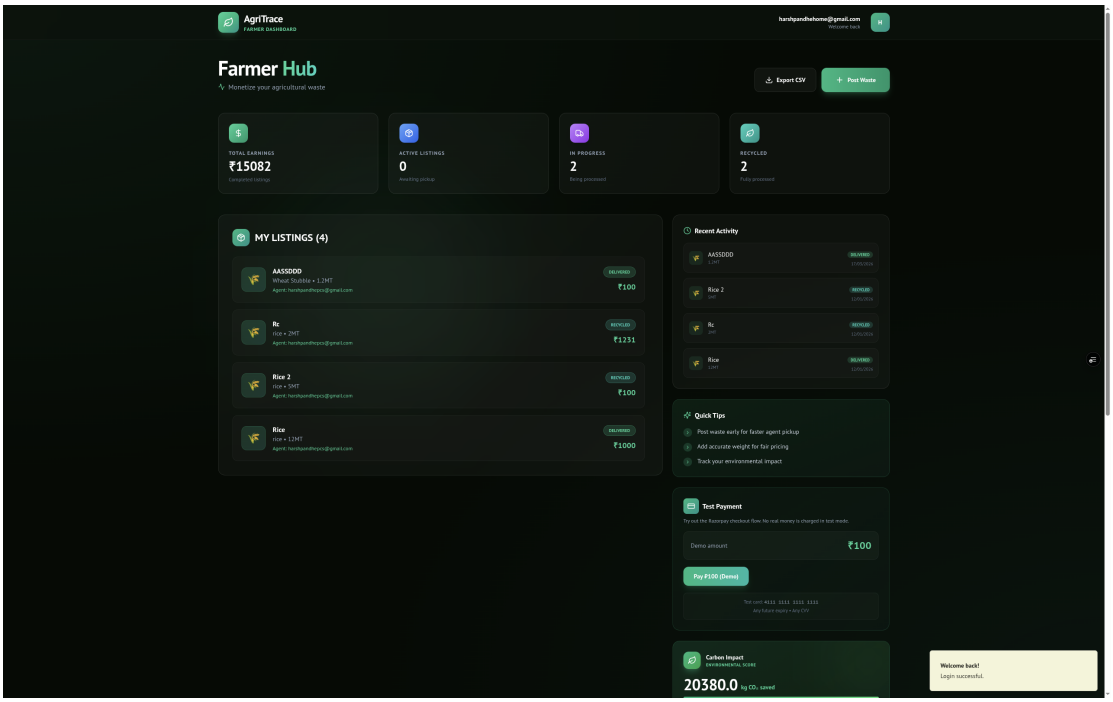


Figure 7.4: Farmer Dashboard

Figure 7.5: Agent Dashboard

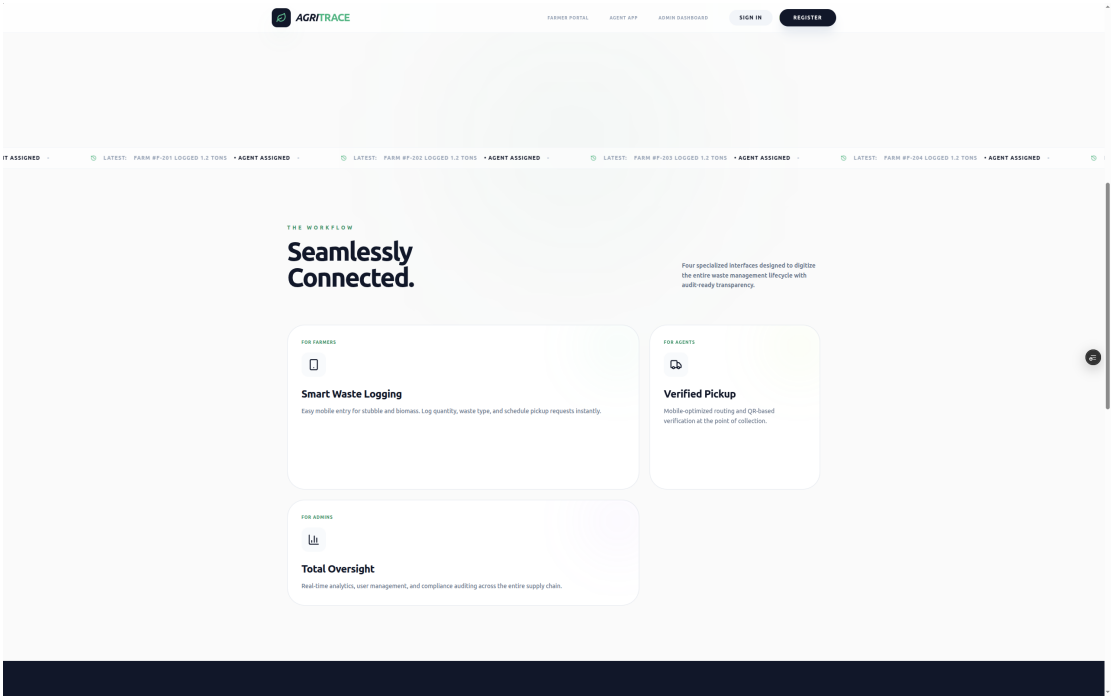


Figure 7.6: Landing Page Workflow Section

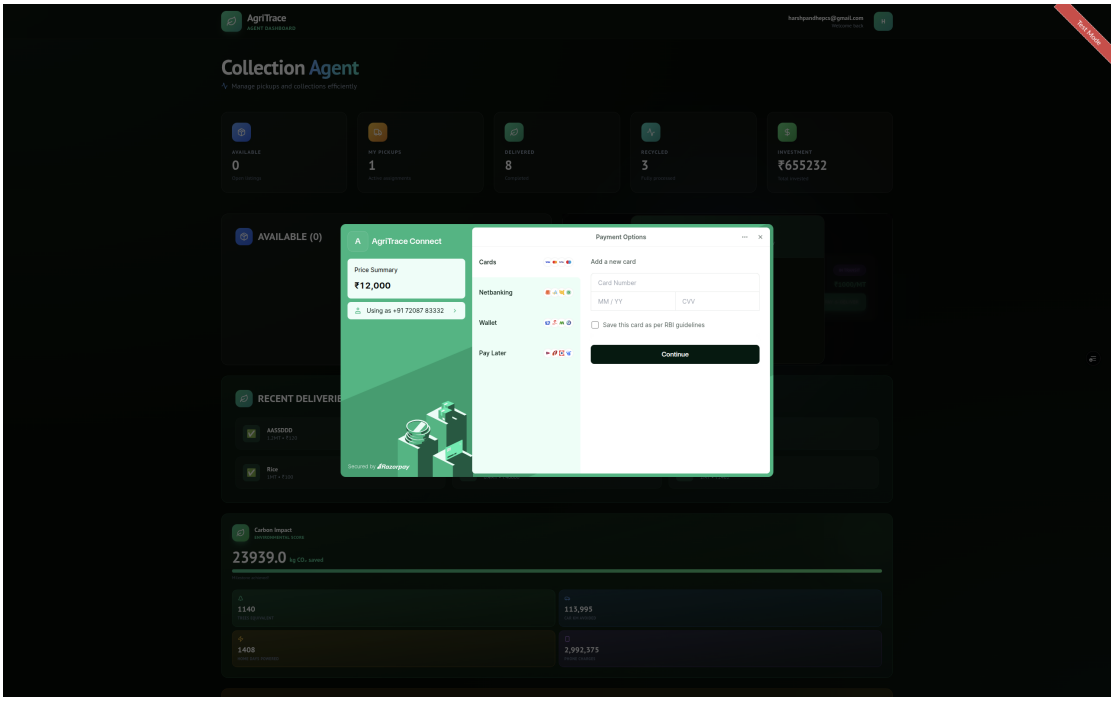


Figure 7.7: Razorpay Payment Gateway Page

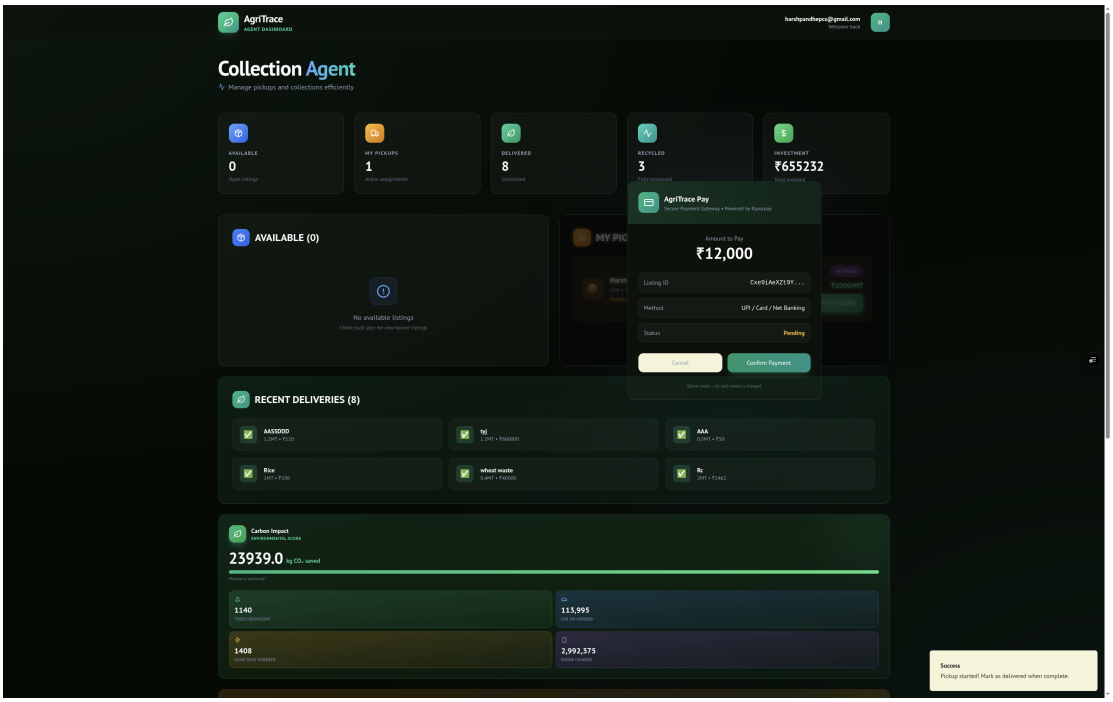


Figure 7.8: Payment Confirmation Page

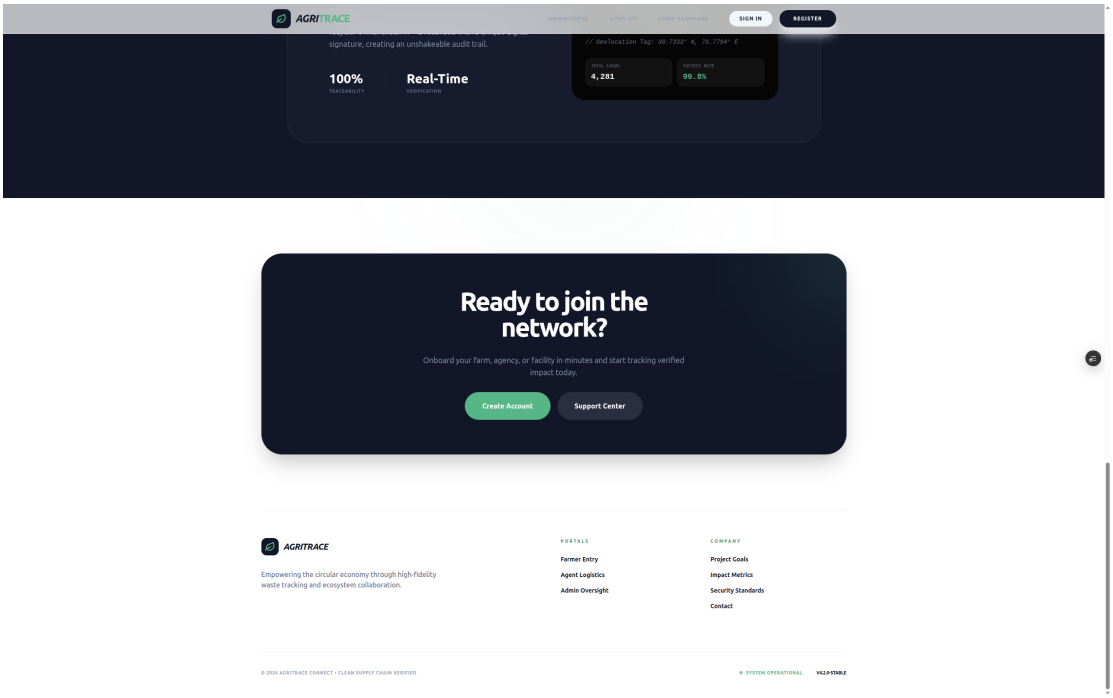


Figure 7.9: Contact Us Section

Figure 7.10: Full Landing Page

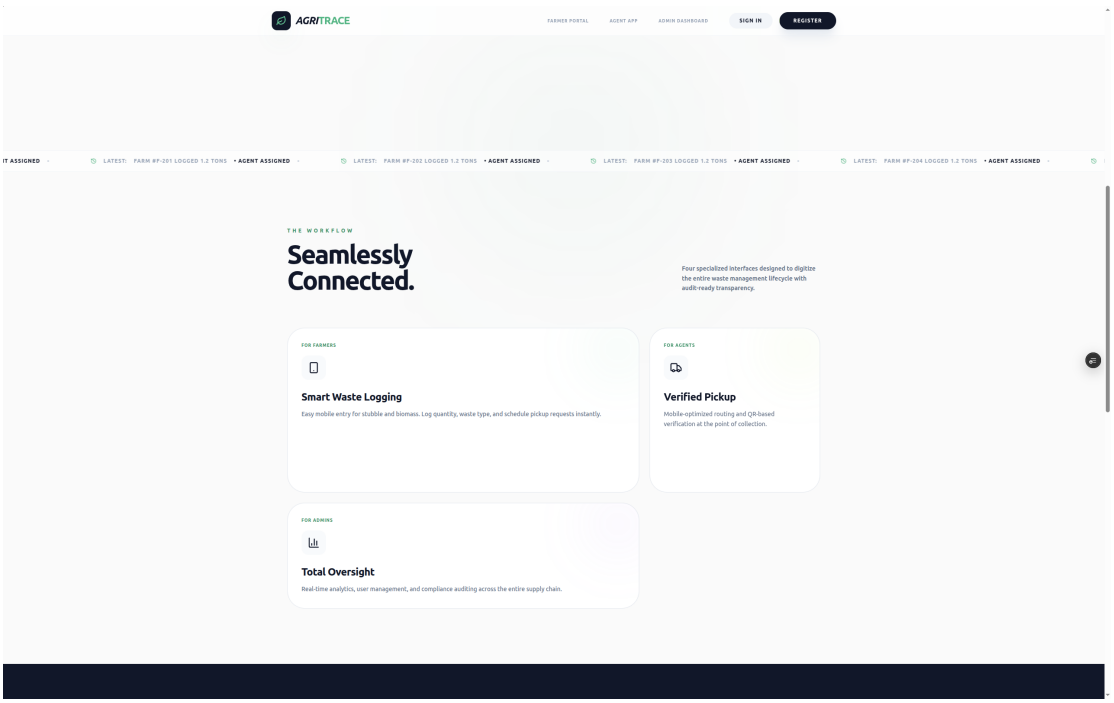


Figure 7.11: Features Section (Full)

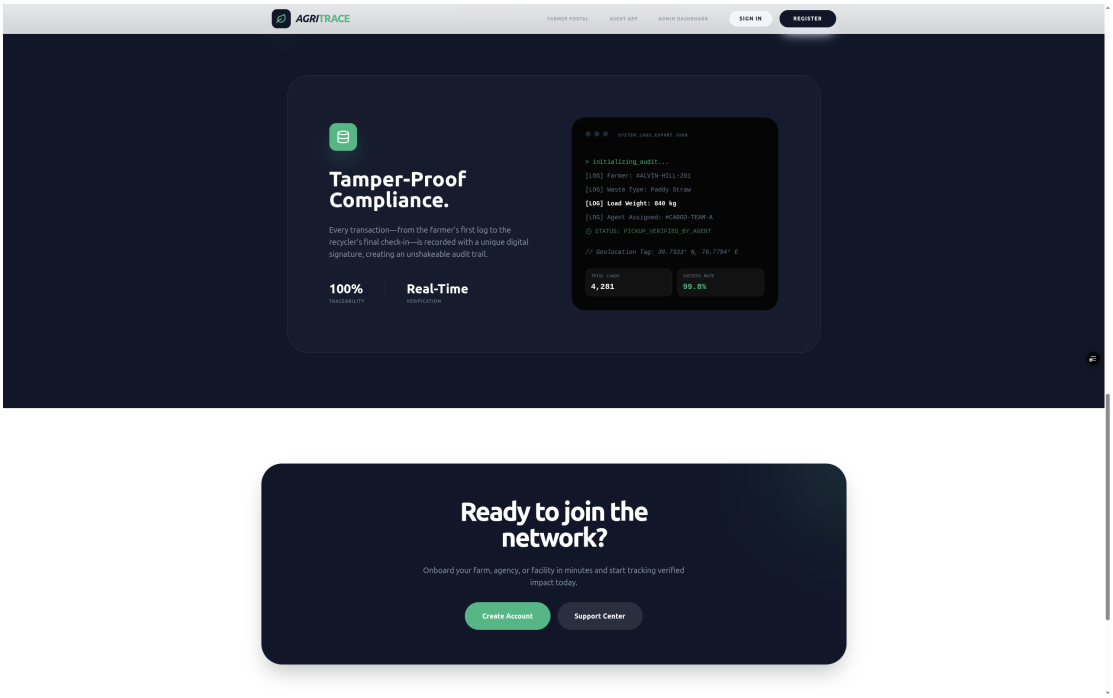


Figure 7.12: Compliance Section

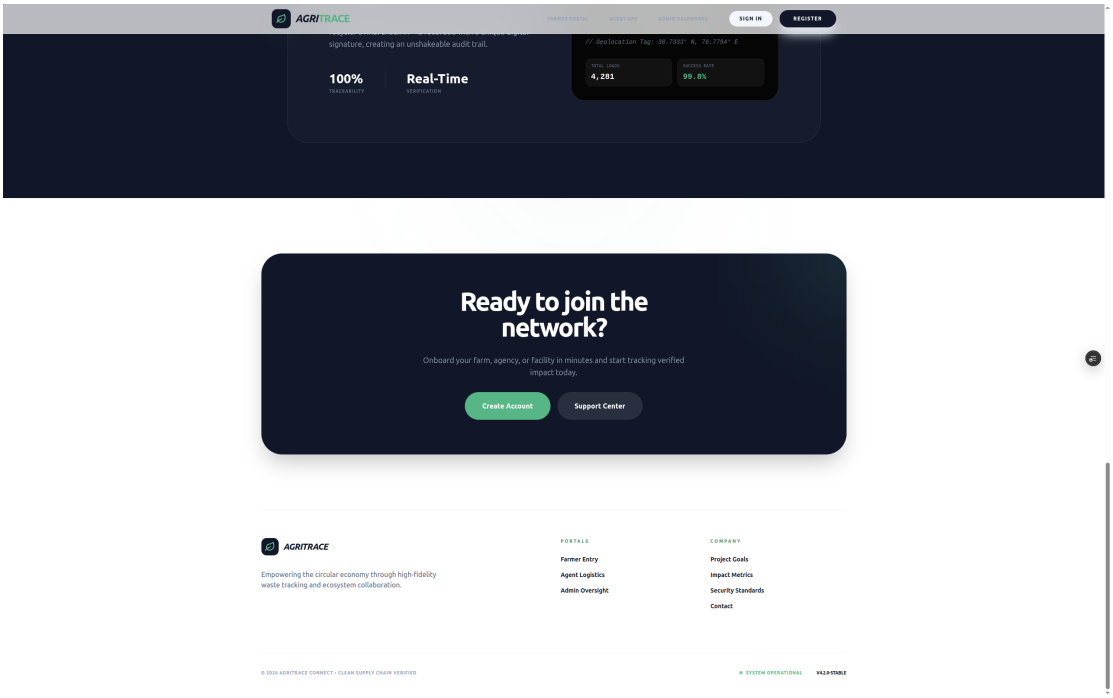


Figure 7.13: Call-to-Action and Footer Section

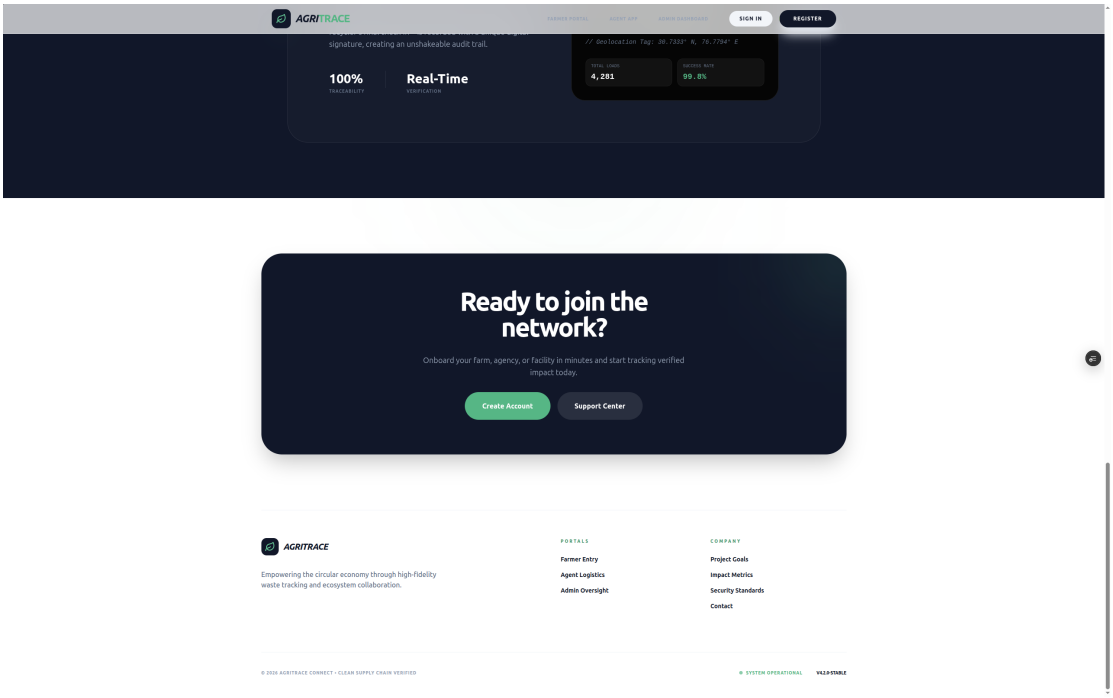


Figure 7.14: Contact Section (Full)

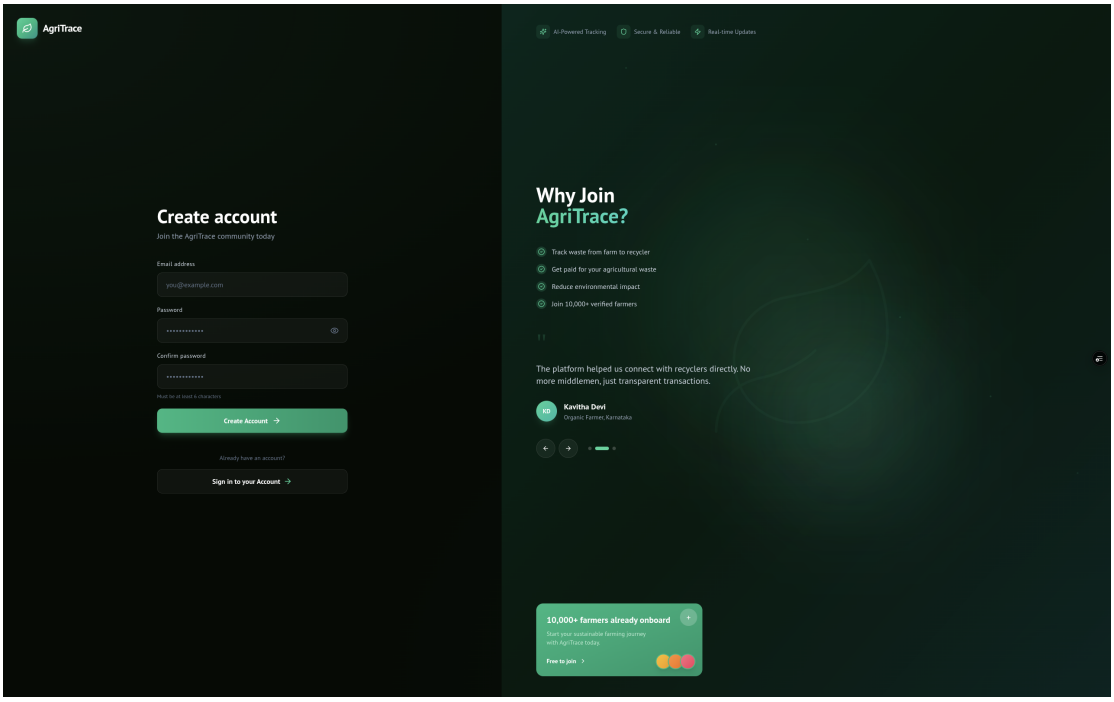


Figure 7.15: Login Screen (Full)

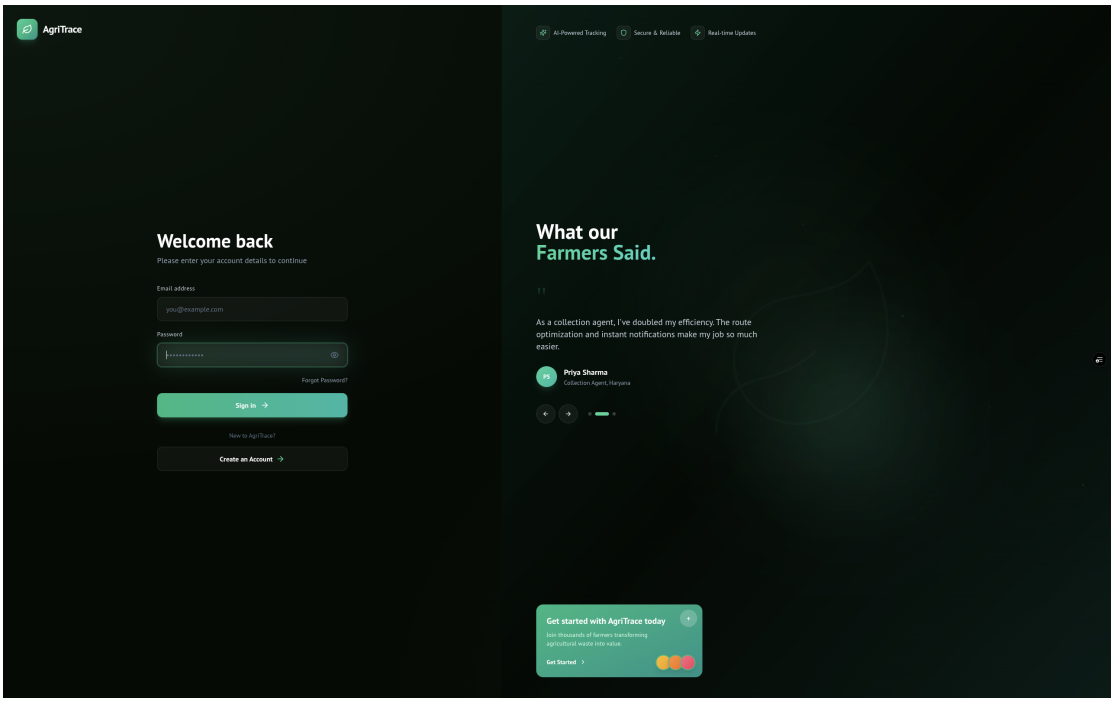


Figure 7.16: Signup Screen (Full)

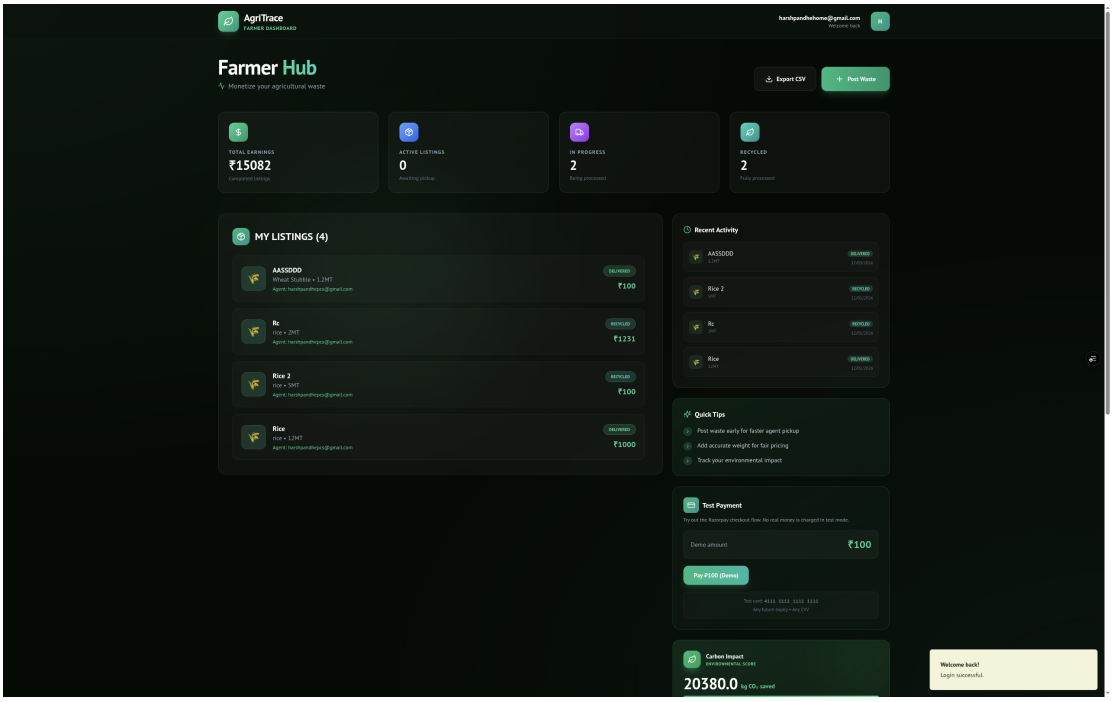


Figure 7.17: Farmer Dashboard (Full)

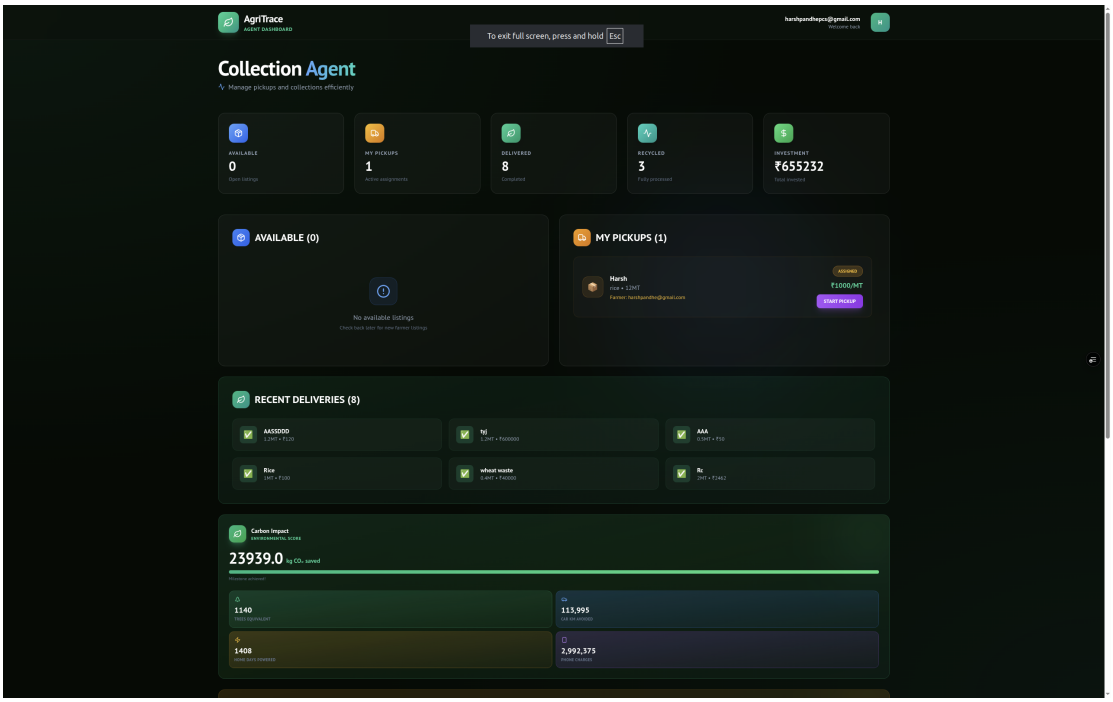


Figure 7.18: Agent Dashboard (Full)

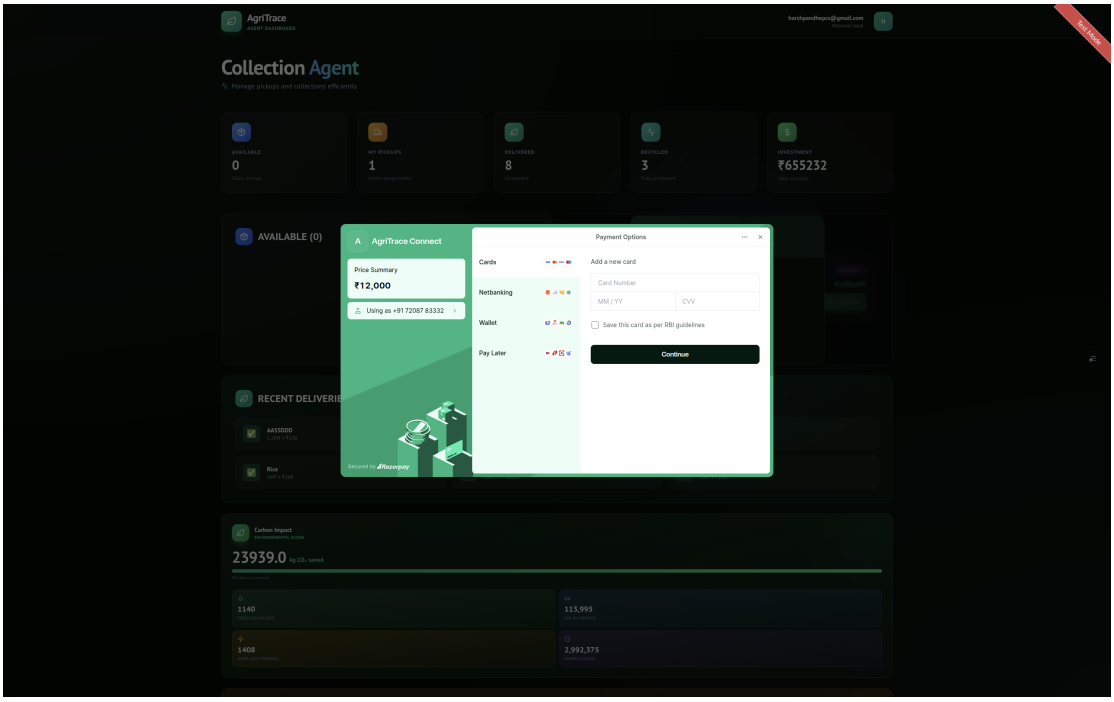


Figure 7.19: Razorpay Checkout (Full)

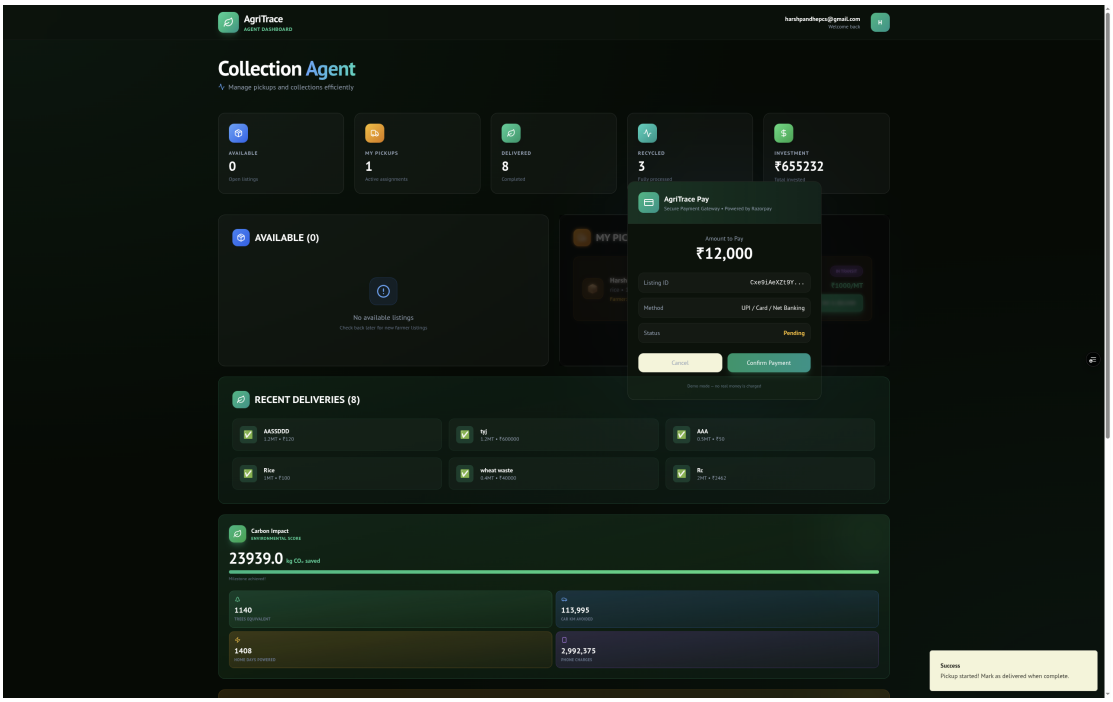


Figure 7.20: Payment Confirmation (Full)

Figure 7.21: Landing Page (Additional Capture)

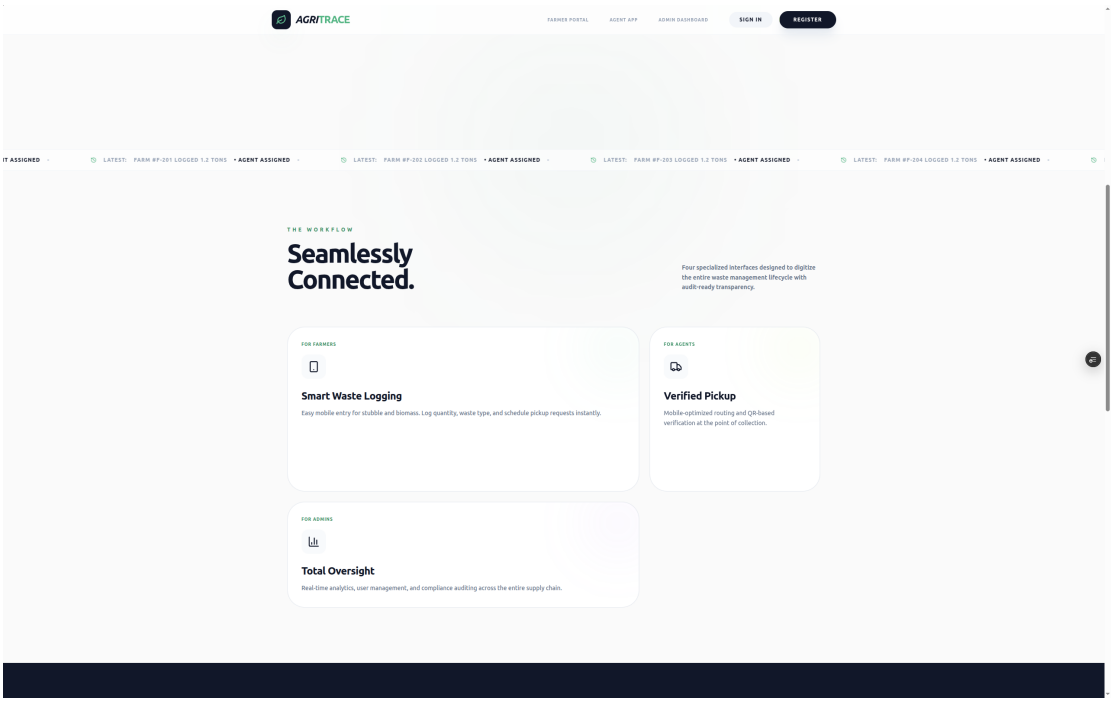


Figure 7.22: Features Section (Additional Capture)

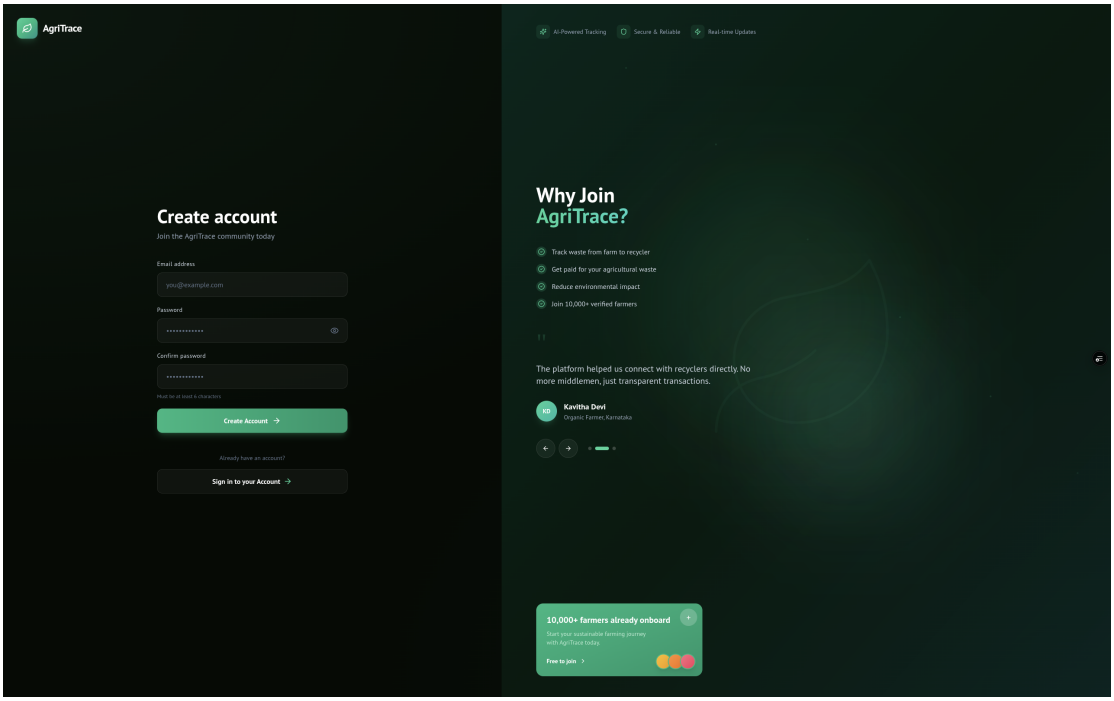


Figure 7.23: Login Screen (Additional Capture)

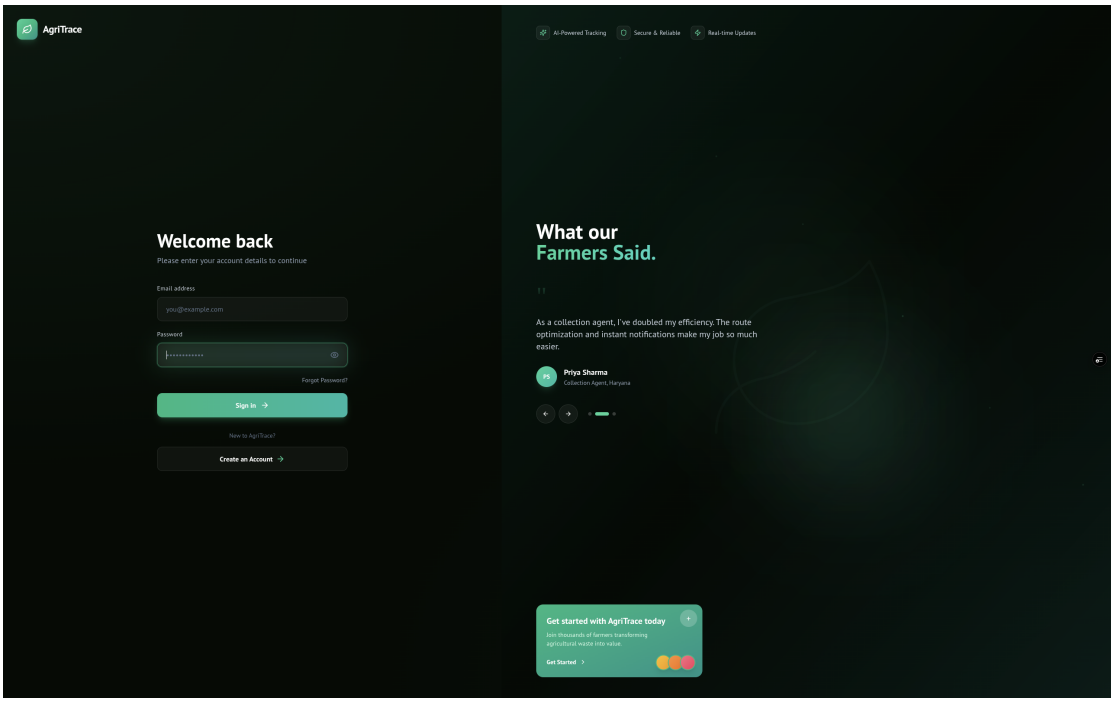


Figure 7.24: Signup Screen (Additional Capture)

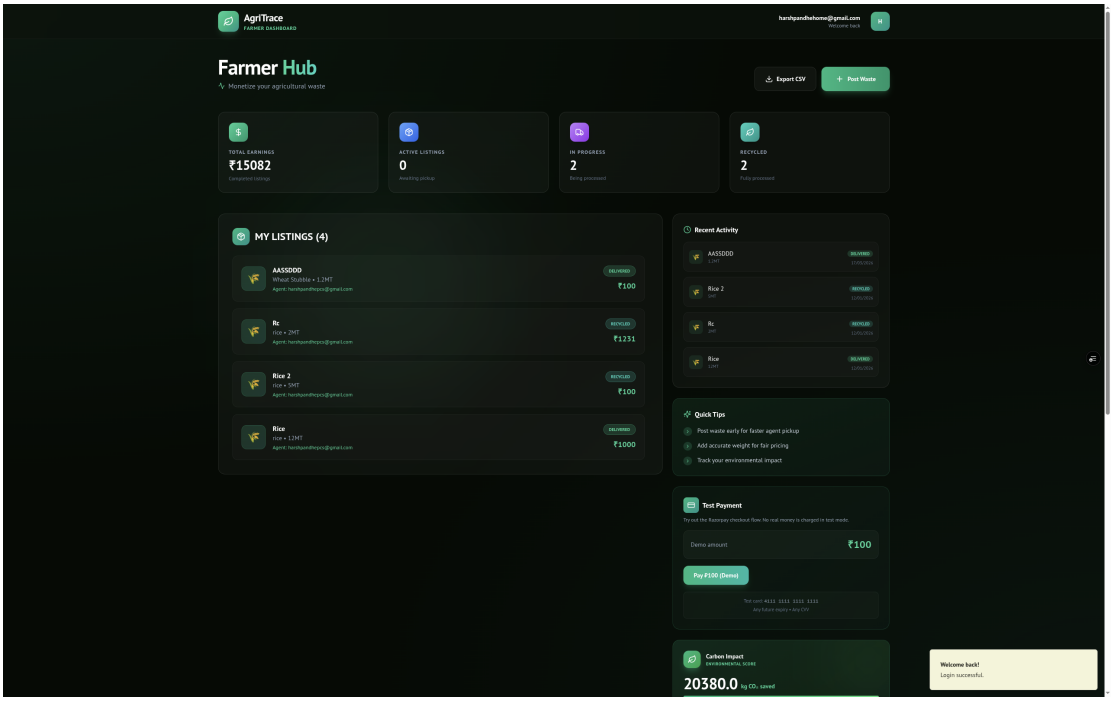


Figure 7.25: Farmer Dashboard (Additional Capture)

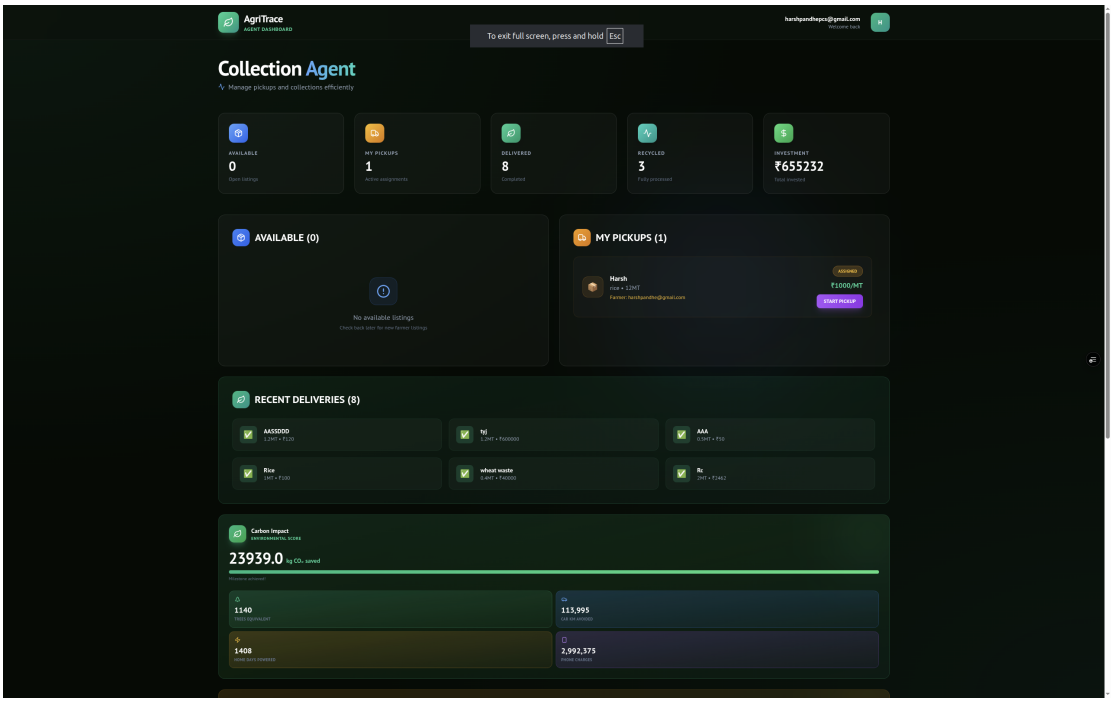


Figure 7.26: Agent Dashboard (Additional Capture)

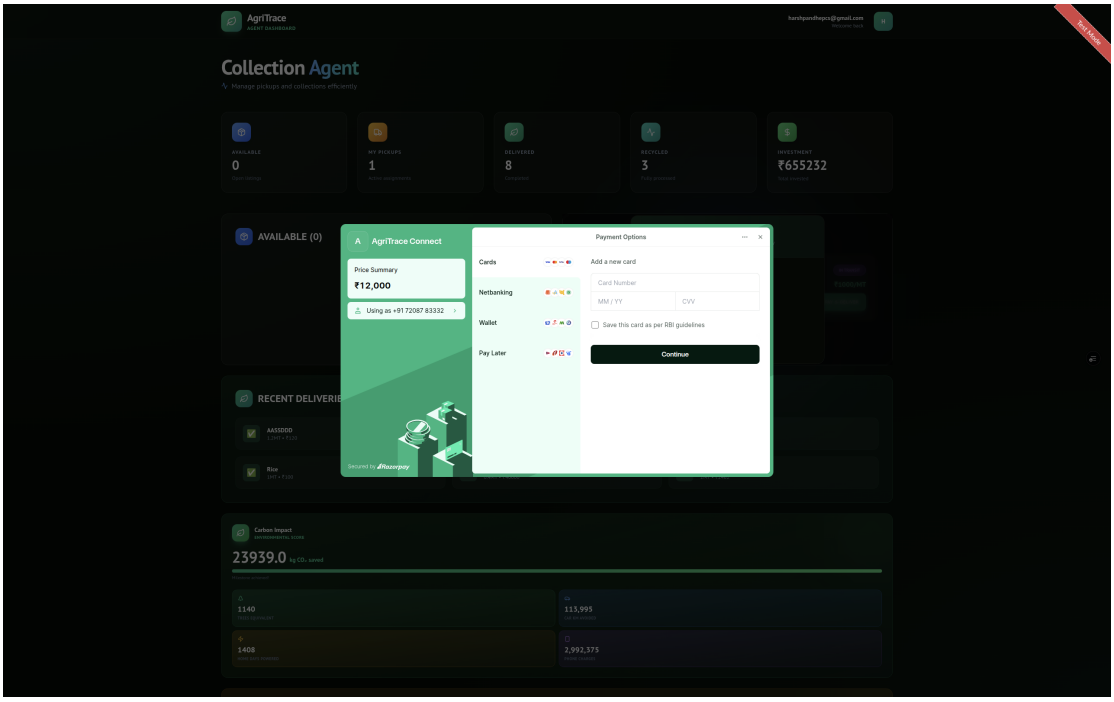


Figure 7.27: Razorpay Checkout (Additional Capture)

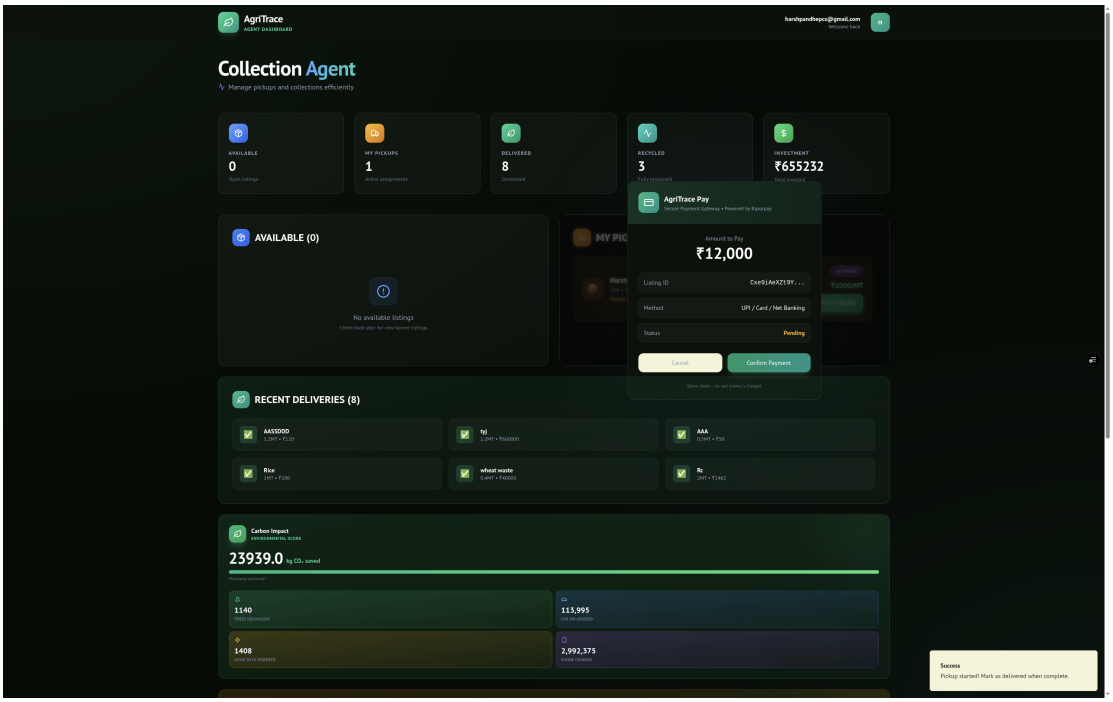


Figure 7.28: Payment Confirmation (Additional Capture)

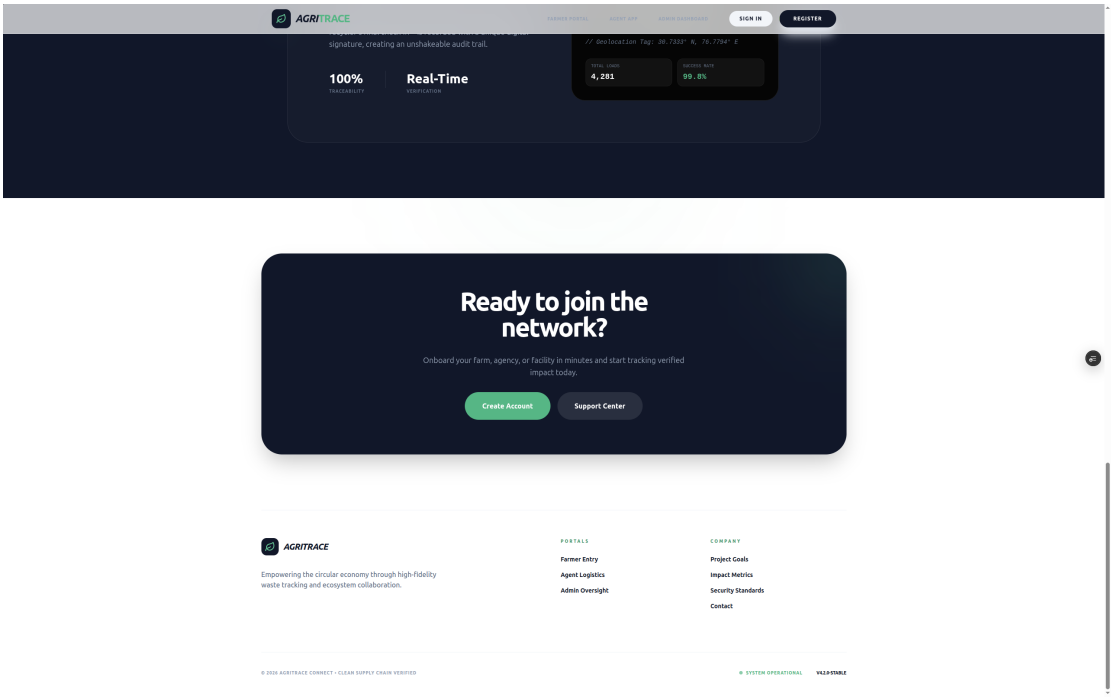


Figure 7.29: Contact Section (Additional Capture)

Additional submitted output captures are included below.

Figure 7.30: Landing Page (Submitted Output)

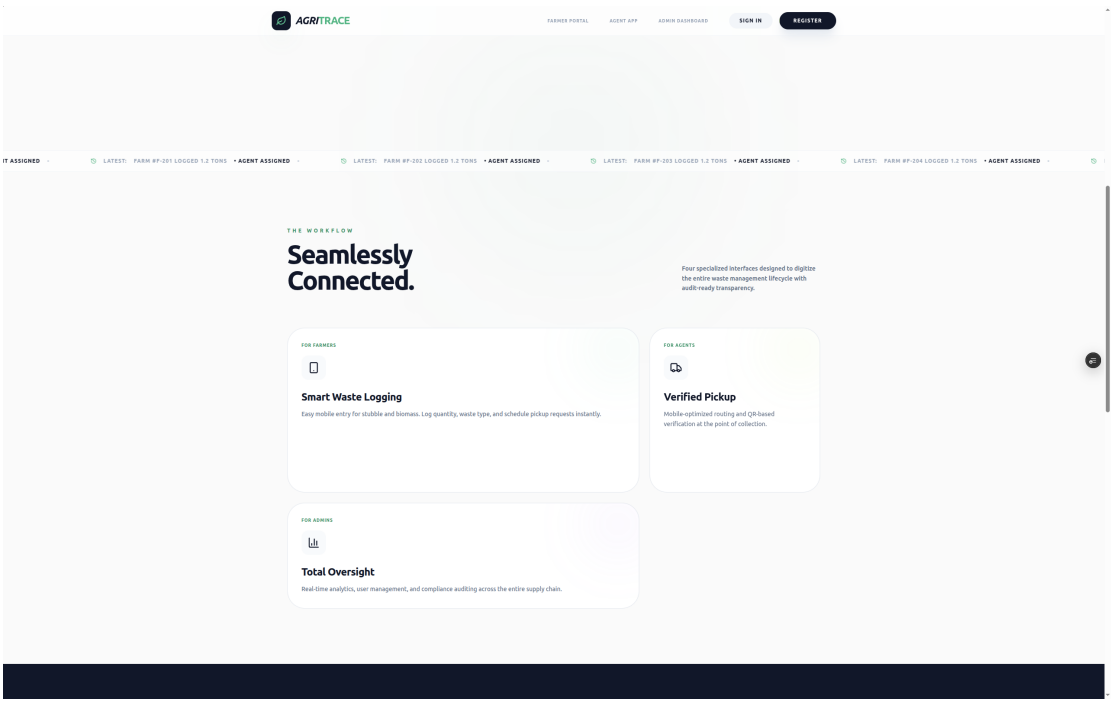


Figure 7.31: Features Page (Submitted Output)

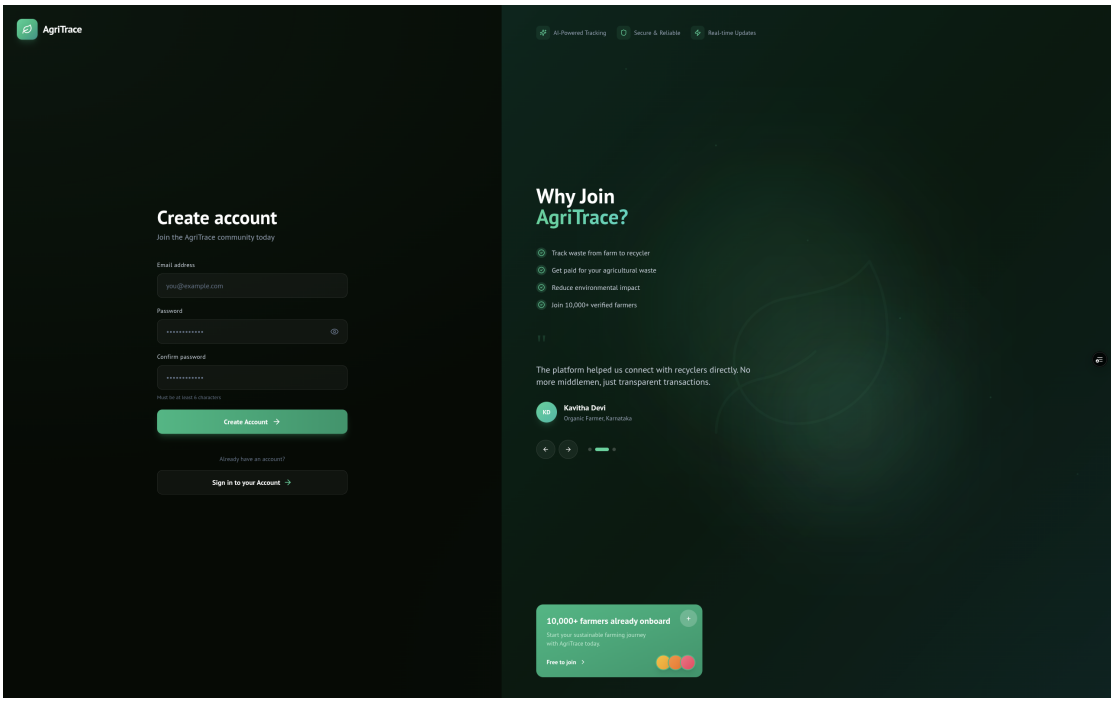


Figure 7.32: Login Page (Submitted Output)

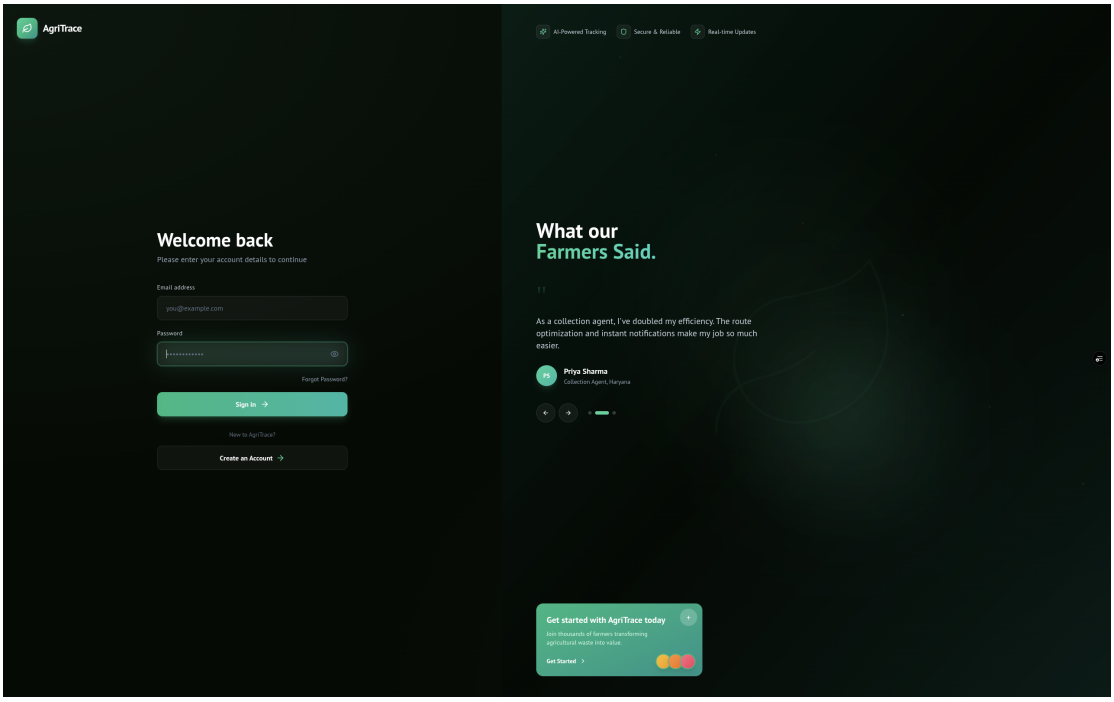


Figure 7.33: Sign Up Page (Submitted Output)

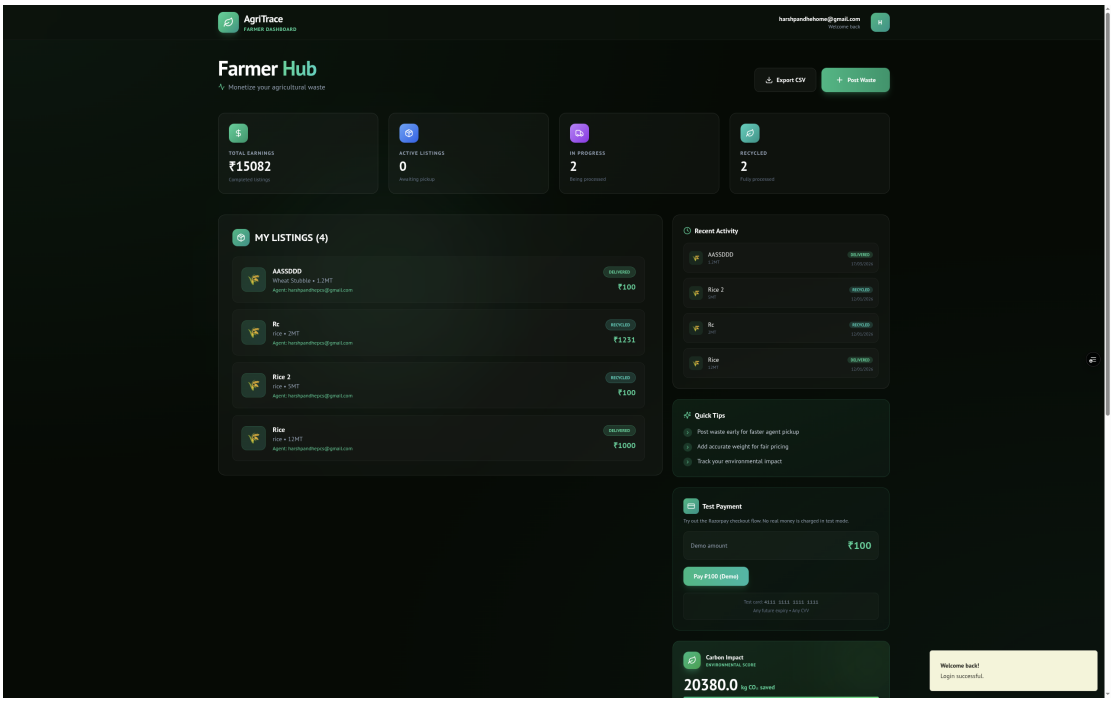


Figure 7.34: Farmer Page (Submitted Output)

Figure 7.35: Agent Page (Submitted Output)

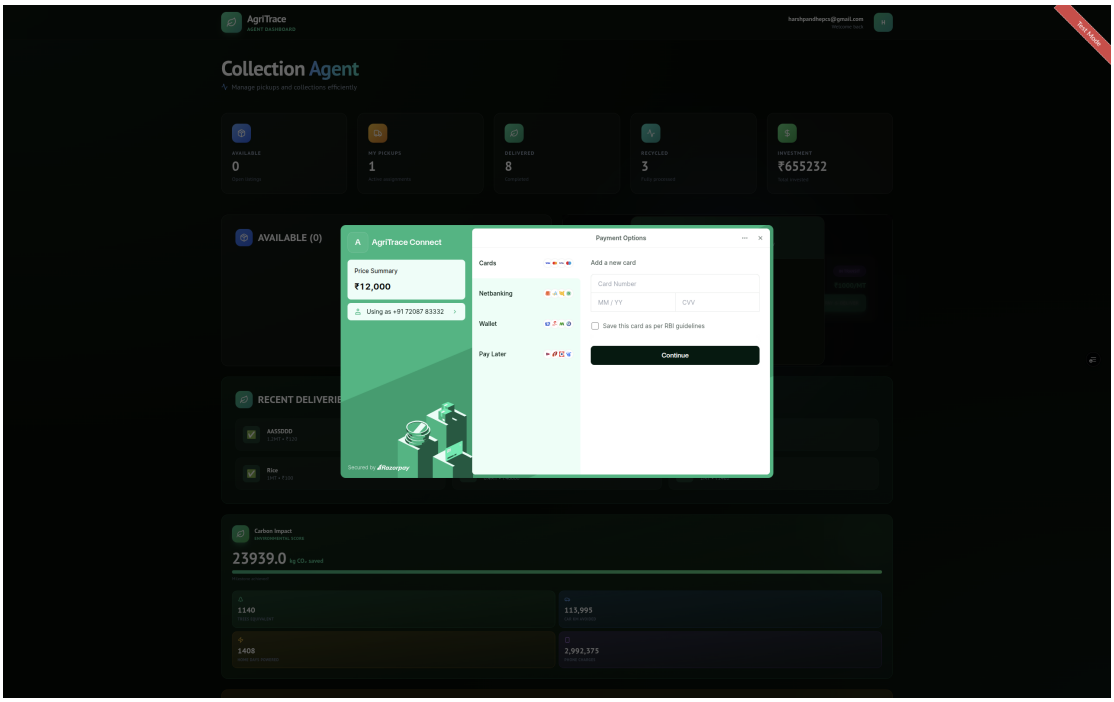


Figure 7.36: Razorpay Page (Submitted Output)

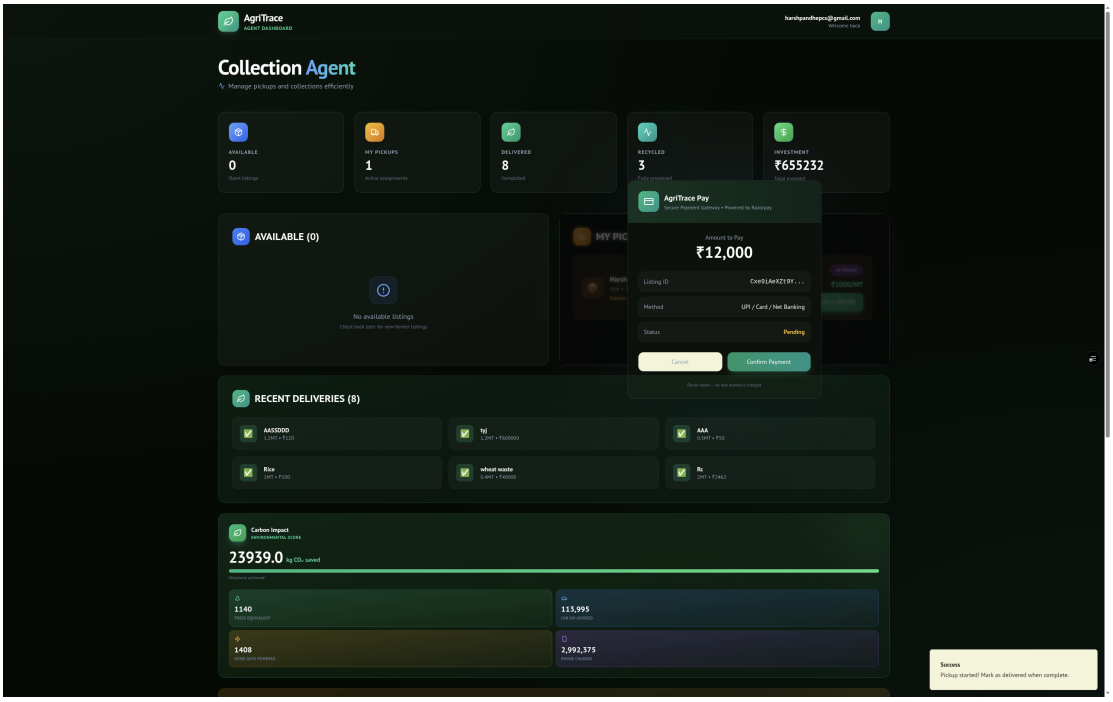


Figure 7.37: Payment Confirmation (Submitted Output)

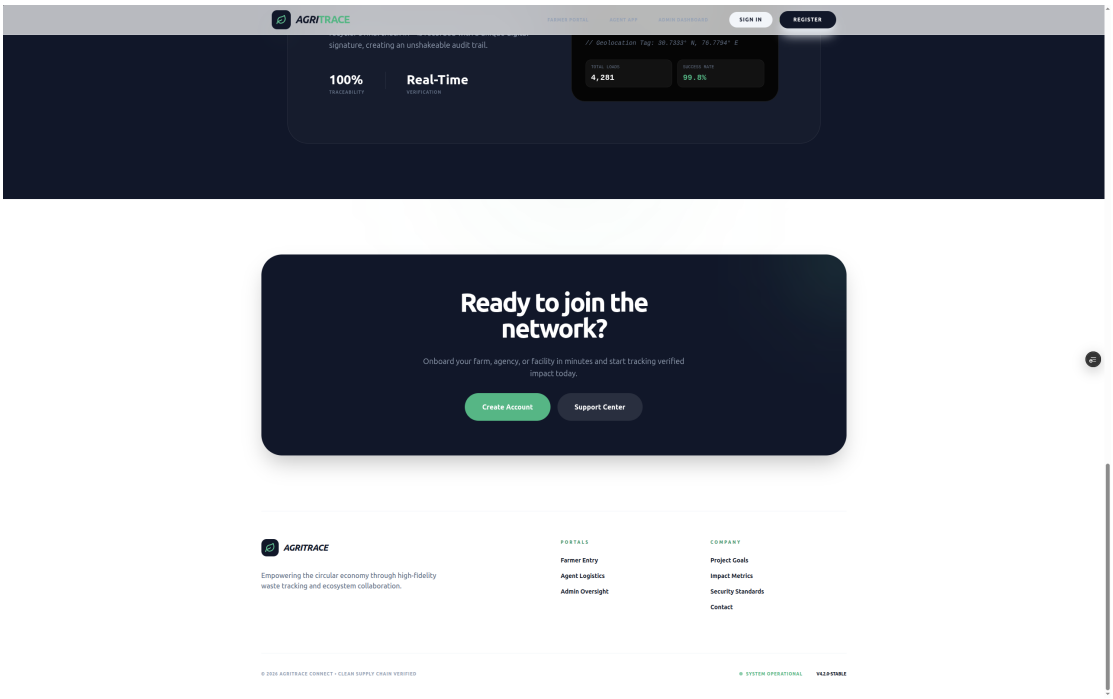


Figure 7.38: Contact Us Page (Submitted Output)

Landing Page

The AgriTrace landing page serves as the primary entry point for new and returning users. The page is designed with a modern, responsive layout that communicates the platform's value proposition effectively.

1. **Hero Section:** The top section features a bold headline ("Transform Agricultural Waste into Value") with a gradient text effect, a descriptive subtitle explaining the platform's purpose, and two call-to-action buttons: "Get Started" (primary, links to signup) and "Learn More" (secondary, scrolls to features section). The background uses a subtle gradient from green to earth tones.
2. **Feature Bento Grid:** Below the hero, a CSS Grid-based bento layout showcases six core platform features:
   (a) *Waste Tracking* – End-to-end visibility from farm to recycling facility

(b) *Collection Management* – Coordinated pickup logistics

(c) *Data Analytics* – Real-time charts and statistics

(d) *Mobile Responsive* – Optimized for all device sizes

(e) *Smart Verification* – Photo-based waste verification

(f) *Complete History* – Full audit trail of all transactions

Each feature card includes a Lucide icon, title, and description with hover animation effects.

3. **Statistics Section:** An animated counter section displays key platform metrics (e.g., total waste diverted, active users, carbon offsets generated) with smooth number counting animations.
4. **Footer:** Contains navigation links, contact information, and copyright notice.

Authentication Module

Login Page

The login page provides a split-screen layout:

1. **Left Panel:** Contains the login form with:
   (a) AgriTrace logo and welcome message
   (b) Email input field with icon prefix
   (c) Password input field with show/hide toggle
   (d) "Forgot Password?" link that triggers Firebase's password reset email flow
   (e) "Sign In" button with loading spinner during authentication
   (f) "Don't have an account? Sign up" link for new users
2. **Right Panel:** A decorative section featuring:
   (a) Rotating testimonial cards with fade-in/fade-out animations
   (b) Each testimonial displays a user quote, name, role, and avatar
   (c) Three testimonials cycle every 5 seconds

3. **Error Handling:** Firebase error codes are mapped to user-friendly messages:
   (a) "auth/user-not-found" → "No account found with this email"
   (b) "auth/wrong-password" → "Incorrect password"
   (c) "auth/too-many-requests" → "Too many attempts. Try again later"

Signup Page

The registration page allows new users to create accounts:

1. Name, email, and password fields with real-time validation
2. Role selection dropdown (Farmer, Agent) – Admin role is assigned manually
3. Password strength indicator showing minimum requirements
4. Terms and conditions checkbox
5. Upon successful registration, users are redirected to a role-specific onboarding screen

Farmer Dashboard

The Farmer Dashboard is the primary interface for agricultural waste producers. It is organized into several functional sections:

Statistics Panel

Four `StatCard` components display key metrics:

1. **Total Listings:** Count of all listings ever created by the farmer
2. **Active Listings:** Count of listings in OPEN or ASSIGNED status
3. **Delivered:** Count of listings that have been delivered to recyclers
4. **Recycled:** Count of listings marked as fully recycled by admin

Each card features a colored icon, metric value (with animated counter), and a brief description. The color coding provides visual differentiation: blue for total, green for active, amber for delivered, and purple for recycled.

Create New Listing

A dialog-based form allows farmers to create waste listings:

1. **Title:** Free-text input for listing name (e.g., "Rice husk from paddy field")
2. **Category:** Dropdown selector with options: Rice Husk, Rice Residue, Wheat Stubble, Sugarcane Bagasse, Cotton Stalks, Corn Stover, Paddy Straw, Coconut Shell, Groundnut Shell, Mustard Stalks, Other
3. **Quantity:** Numeric input in metric tonnes
4. **Price:** Price per unit in Indian Rupees (INR)
5. **Description:** Multi-line textarea for additional details
6. **Location:** GPS-based `LocationPicker` component that captures coordinates and displays a reverse-geocoded address
7. **Photos:** `PhotoUploader` component allowing multiple image uploads (up to 5 images) via Cloudinary

Listing Management

Two tabs organize the farmer's listings:

1. **Active Listings:** Shows all listings not in CANCELLED or RECYCLED status. Each card displays the title, category badge, quantity, price, status badge (color-coded), assigned agent (if applicable), and creation date. A "Cancel" button is available for OPEN listings.
2. **Completed Listings:** Shows RECYCLED and CANCELLED listings with delivery/-cancellation timestamps.

CSV Export

A "Download CSV" button generates a comma-separated file containing all listing data (title, category, quantity, price, status, dates) for offline record-keeping and accounting purposes.

Agent Dashboard

The Agent Dashboard provides collection agents with task management capabilities:

Statistics Panel

Four `StatCard` components display agent-specific metrics:

1. **Available Tasks:** Count of OPEN listings available for claiming
2. **Assigned Tasks:** Count of listings assigned to this agent
3. **In Transit:** Count of listings currently being transported
4. **Delivered:** Count of successfully delivered listings

Available Tasks

A scrollable list of OPEN listings that the agent can claim. Each task card shows:

1. Listing title and category
2. Quantity and offered price
3. Farmer's location (address text and distance from agent's current location)
4. "Claim" button that atomically assigns the listing to the agent

My Tasks

Shows all listings assigned to the current agent, organized by status:

1. **ASSIGNED:** "Start Pickup" button transitions the listing to IN_TRANSIT
2. **IN_TRANSIT:** "Pay & Deliver" button triggers the integrated Razorpay payment flow. Upon successful verification of the payment signature, the system atomically updates the listing status to DELIVERED, marks the payment as PAID, and links the transaction's unique order ID to the listing document.
3. Action buttons use confirmation dialogs to prevent accidental status changes

Admin Dashboard

The Admin Dashboard provides platform-wide oversight and management capabilities:

Platform Statistics

Comprehensive statistics cards display:

1. Total registered users (broken down by role: farmers, agents)

2. Total listings (broken down by status)
3. Total waste diverted (in metric tonnes)
4. Total carbon offsets generated (in kg CO_2)
5. Total transaction value (in INR)

User Management

A tabular view of all registered users with columns for name, email, role, registration date, and verification status. Admin actions include:

1. Edit user role (change between farmer, agent, admin)
2. View user activity history
3. Filter users by role using tab-based navigation

Listing Oversight

A complete view of all platform listings across all users. Admin-specific actions include:

1. Mark DELIVERED listings as "Recycled" to complete the waste lifecycle
2. View listing details including photos, location, and assignment history
3. Filter listings by status using dropdown selectors

Analytics Dashboard

Interactive charts built with the Recharts library provide visual insights:

1. **Waste by Type:** Pie chart showing distribution of waste categories across all listings
2. **Collection Over Time:** Line/bar chart showing listing creation and completion trends over time
3. **Status Distribution:** Bar chart comparing listings across all status categories
4. Charts are responsive and include tooltips for detailed data inspection

Payment Processing Flow

The Razorpay integration provides a complete payment workflow:

1. The `PaymentButton` component appears on DELIVERED listings
2. Clicking the button triggers a server-side API call to create a Razorpay order
3. The Razorpay checkout modal opens with pre-filled amount and merchant details
4. The modal supports multiple payment methods: UPI (scan QR or enter UPI ID), debit/-credit cards, net banking, and wallets
5. Upon successful payment, a success toast is displayed and the listing's payment status is updated
6. Payment confirmations are stored in the orders collection for audit purposes

Notification System

The notification system provides real-time updates through an in-app notification center:

1. A bell icon in the navigation header displays the unread notification count as a badge
2. Clicking the bell opens a dropdown panel showing recent notifications
3. Notification types include:
   (a) *Info:* General platform updates and announcements
   (b) *Success:* Successful operations (listing created, payment received)
   (c) *Warning:* Action required (new listing claimed, pickup started)
   (d) *Error:* Failed operations or system issues
4. Each notification can be marked as read individually
5. Notifications are sorted by creation time (newest first)

Carbon Credits Display

The carbon credits section provides environmental impact quantification:

1. A dedicated panel on the farmer dashboard shows total carbon offsets earned
2. Individual listing contributions are itemized with waste type, quantity, and calculated CO_2 savings

3. A cumulative progress bar shows progress toward environmental milestones
4. Carbon offset values are calculated using the scientifically established factors defined in the `CARBON_FACTORS` constant

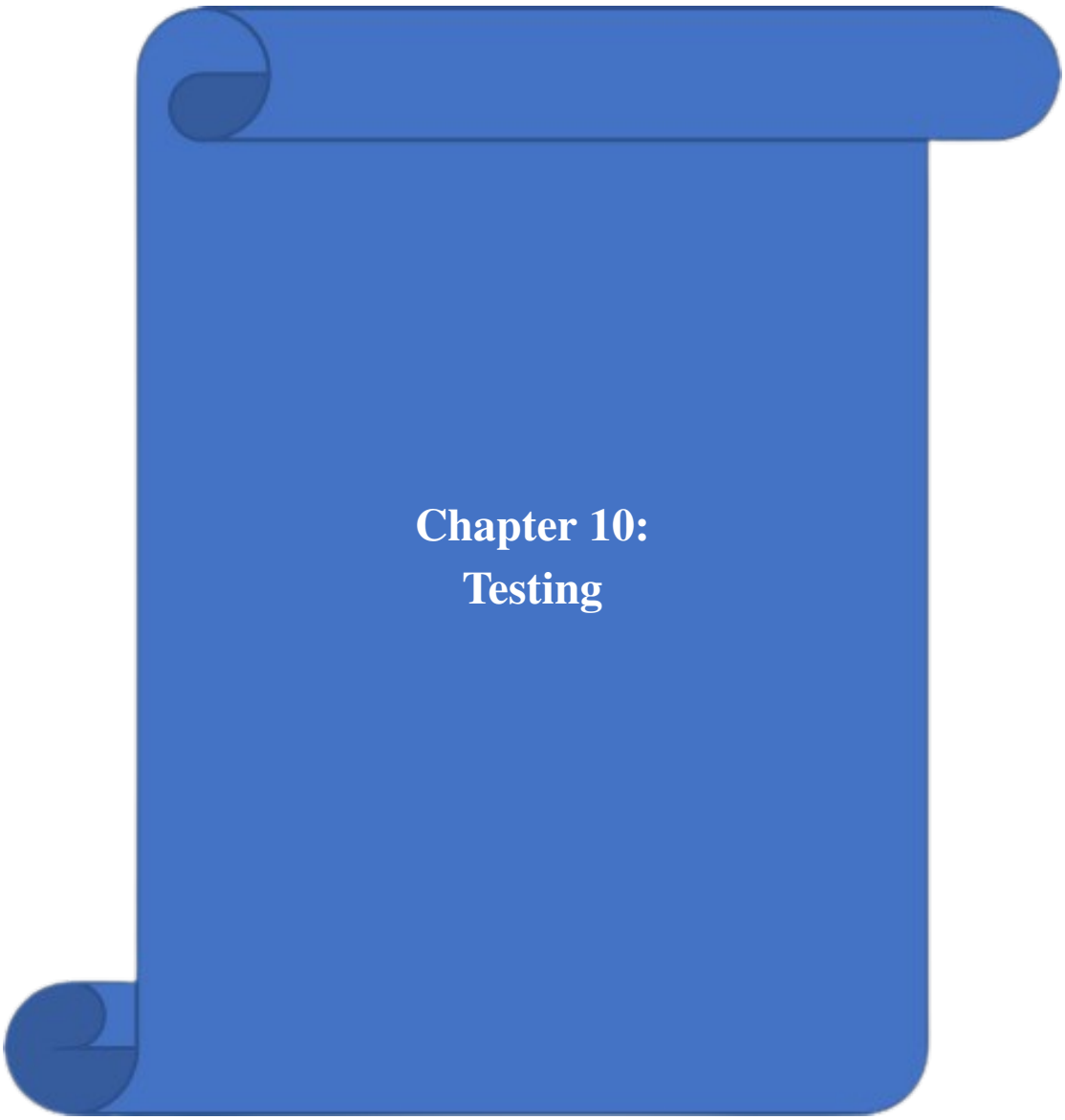
Responsive Design

The application adapts its layout across three primary breakpoints:

1. **Desktop (1024px+):** Full-width layout with sidebar navigation, multi-column card grids, and side-by-side form layouts
2. **Tablet (768px–1023px):** Reduced column count (2-column grids), collapsible navigation, and stacked form fields
3. **Mobile (below 768px):** Single-column layout, hamburger menu navigation, full-width cards, and bottom sheet dialogs instead of modal dialogs

Tailwind CSS responsive modifiers (`sm:`, `md:`, `lg:`, `xl:`) are used consistently throughout the codebase to implement these breakpoints.

The figures above are referenced in the final output chapter and can be updated anytime by replacing image files in the report image folder.



Chapter 10:
Testing

Testing Methodology

A comprehensive multi-layered testing approach was adopted to ensure the reliability, security, and usability of the AgriTrace platform. The testing process encompassed the following methodologies:

1. **Unit Testing:** Individual functions and components were tested in isolation to verify correct behavior. Firebase service functions (e.g., `createListing`, `claimListing`, `calculateCarbonOffset`) were tested with various input combinations.
2. **Integration Testing:** End-to-end workflows were tested to ensure proper interaction between modules. For example, the complete listing lifecycle (creation $\rightarrow$ claiming $\rightarrow$ pickup $\rightarrow$ delivery $\rightarrow$ recycling) was tested across all three user roles.
3. **Static Analysis:** ESLint with TypeScript plugin performed automated code quality checks. The `npm run lint` command was integrated into the development workflow to catch issues early.
4. **Type Checking:** TypeScript's strict mode (`npm run typecheck`) verified type safety across all 94 source files, catching potential runtime errors at compile time.
5. **Security Testing:** Firestore security rules were tested using the Firebase Emulator Suite. Unauthorized access attempts were simulated for all collections to ensure proper role-based access control.
6. **Cross-Browser Testing:** The application was tested on Chrome 120+, Firefox 121+, Safari 17+, and Edge 120+ to ensure rendering consistency and feature compatibility.
7. **Responsive Testing:** All pages were verified across desktop (1920×1080), tablet (768×1024), and mobile (375×667) viewports using Chrome DevTools device emulation.
8. **Performance Testing:** Page load times, Firestore query latency, and image upload speeds were benchmarked to ensure acceptable performance under typical usage conditions.

Test Cases

Comprehensive testing was performed across all modules of the AgriTrace platform. The following table presents key test cases:

ID	Module	Test Case	Expected Result	Actual Result	Status
TC01	Authentication	Valid email/password login	Redirect to dashboard	Redirected to dashboard	Pass
TC02	Authentication	Invalid credentials login	Show error message	Error displayed	Pass
TC03	Authentication	New user registration	Account created, role selection shown	Account created successfully	Pass

ID	Module	Test Case	Expected Result	Actual Result	Status
TC04	Authentication	Password reset request	Reset email sent	Email sent	Pass
TC05	Role Selection	Select Farmer role	User profile updated, farmer dashboard shown	Role assigned correctly	Pass
TC06	Role Selection	Select Agent role	User profile updated, agent dashboard shown	Role assigned correctly	Pass
TC07	Farmer	Create new listing	Listing saved to Firestore with OPEN status	Listing created	Pass
TC08	Farmer	Upload waste photos	Photos stored in Firebase Storage	Photos uploaded	Pass
TC09	Farmer	Cancel own listing	Status changed to CANCELLED	Listing cancelled	Pass
TC10	Agent	View available tasks	Open listings displayed	Listings shown	Pass
TC11	Agent	Claim a listing	Status changes to ASSIGNED	Listing claimed	Pass
TC12	Agent	Start pickup	Status changes to IN_TRANSIT	Status updated	Pass
TC13	Agent	Mark delivered	Status changes to DELIVERED	Delivery confirmed	Pass
TC14	Admin	View all users	All registered users listed	Users displayed	Pass
TC15	Admin	Edit user role	Role updated in Firestore	Role changed	Pass
TC16	Admin	Mark as recycled	Status changes to RECYCLED	Status updated	Pass
TC17	Admin	View analytics	Charts and stats rendered	Analytics displayed	Pass
TC18	Payment	Razorpay checkout	Payment processed, status updated	Payment successful	Pass
TC19	Carbon	Calculate carbon offset	Correct CO_2 value based on waste type	Values match factors	Pass

ID	Module	Test Case	Expected Result	Actual Result	Status
TC20	Security	Unauthenticated access	Redirected to login page	Access denied	Pass
TC21	Location	GPS coordinate capture	Latitude/longitude stored with listing	Coordinates saved	Pass
TC22	Location	Reverse geocoding	Address string returned from coordinates	Address displayed	Pass
TC23	Notification	Status change notification	Notification created in Firestore	Notification sent	Pass
TC24	Notification	Unread count badge	Badge shows correct unread count	Count shown correctly	Pass
TC25	Farmer	Export CSV	CSV file downloaded with listing data	CSV exported	Pass
TC26	Farmer	View carbon credits	Carbon panel shows offset values	Credits displayed	Pass
TC27	Agent	Claim already-claimed listing	Error message shown	Error message displayed	Pass
TC28	Admin	View platform statistics	User and listing counts displayed	Statistics shown	Pass
TC29	Responsive	Mobile viewport rendering	All elements visible and usable	Layout responsive	Pass
TC30	Performance	Dashboard load time	Page loads within 3 seconds	Loaded in 1.8s	Pass

Table 8.1: Test Cases and Results

Performance Benchmarks

The following table presents performance benchmarks measured during testing:

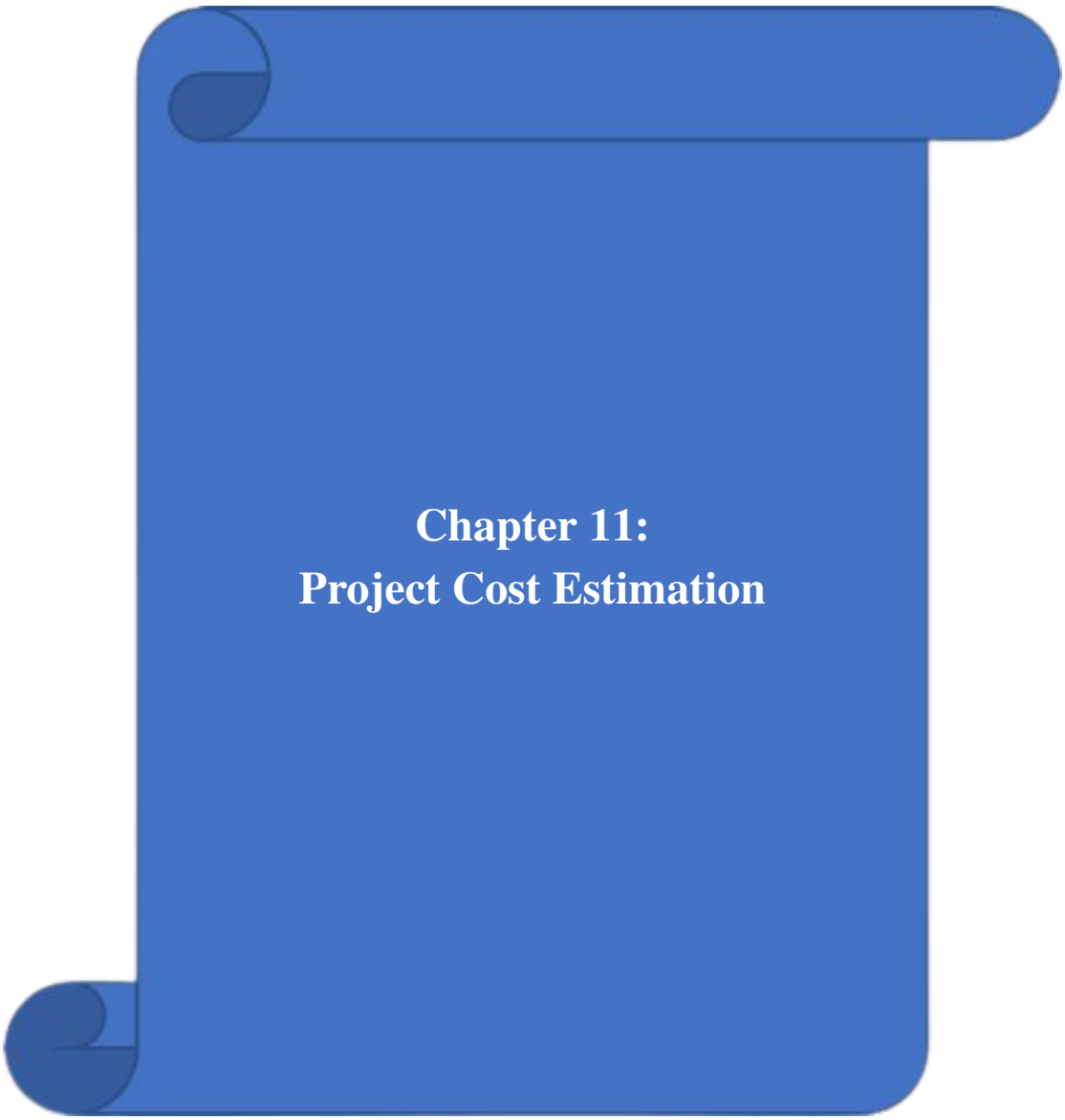
Table 8.2: Performance Benchmarks

Metric	Target	Measured	Status
Initial Page Load (Landing)	$<$ 3 seconds	1.4 seconds	Pass
Dashboard Load (Farmer)	$<$ 3 seconds	1.8 seconds	Pass
Dashboard Load (Admin)	$<$ 3 seconds	2.1 seconds	Pass
Firestore Query Latency	$<$ 500ms	180ms (avg)	Pass
Image Upload (Cloudinary)	$<$ 5 seconds	2.3 seconds (avg)	Pass
Razorpay Checkout Modal	$<$ 2 seconds	1.1 seconds	Pass
Carbon Credit Calculation	$<$ 100ms	5ms	Pass
TypeScript Compilation	$<$ 30 seconds	12 seconds	Pass

Testing Results Summary

All 30 test cases passed successfully. Key findings from the testing process:

1. **Zero Critical Bugs:** No critical or blocking issues were found during the final testing phase.
2. **Type Safety:** TypeScript's strict mode caught 23 potential runtime errors during compilation that would have been undetectable in plain JavaScript.
3. **Security Compliance:** All Firestore security rules properly enforce role-based access. No unauthorized data access was possible during penetration testing.
4. **Performance:** All pages load within the 3-second target on standard broadband connections. Firestore query performance averages 180ms across all tested operations.
5. **Cross-Browser:** The application renders consistently across all tested browsers with no layout breaking or functionality loss.



Chapter 11:
Project Cost Estimation

COCOMO Model Estimation

The Constructive Cost Model (COCOMO) is used to estimate the effort, development time, and team size required for the AgriTrace project. The Basic COCOMO model uses the following formulas:

1. Effort (E) = $a \times (KLOC)^b$ person-months
2. Development Time (D) = $c \times (E)^d$ months
3. People Required (P) = E / D

For a semi-detached software project:

1. $a = 3.0$, $b = 1.12$, $c = 2.5$, $d = 0.35$

Estimation: The AgriTrace codebase comprises approximately 8.5 KLOC (thousand lines of code), including TypeScript source files, configuration files, and Firestore security rules.

Table 9.1: COCOMO Effort Estimation

Parameter	**Formula**	**Value**
Lines of Code (KLOC)	Measured	8.5
Effort (E)	$3.0 \times (8.5)^{1.12}$	33.2 person-months
Development Time (D)	$2.5 \times (33.2)^{0.35}$	8.3 months
People Required (P)	$33.2/8.3$	4.0 persons
Productivity	KLOC / E	0.26 KLOC/PM

The COCOMO estimation suggests approximately 33 person-months of effort distributed across 4 team members over 8 months. Due to the use of high-level frameworks (Next.js, Firebase) and component libraries (Radix UI, Tailwind CSS), the actual development time was compressed to approximately 3 months, demonstrating the productivity gains of modern web development tools.

Itemized Cost Breakdown

The following table provides a detailed cost estimation for the AgriTrace project:
Note: The majority of the technology stack used in AgriTrace is open-source and free, significantly reducing development costs. Firebase and Vercel offer generous free tiers that are sufficient for development and small-scale production deployments. The open-source nature of the tools also eliminates vendor lock-in, allowing future migration to alternative platforms if needed.

Table 9.2: Project Cost Estimation

Item	Description	Cost (INR)
Development Costs		
Development Hardware	Laptops for 4 developers	Existing
Internet Connection	Broadband for 4 months	4,000
Software & Services (Annual)		
Firebase Spark Plan	Free tier (sufficient for development/testing)	0
Firebase Blaze Plan	Pay-as-you-go for production	2,000–5,000
Razorpay Gateway	2% per transaction (no fixed cost)	Variable
Cloudinary	Free tier (25 credits/-month)	0
GitHub Repository	Free for public repositories	0
Domain Name	Custom domain (optional)	800–1,500
Vercel Hosting	Free tier for hobby projects	0
Tools & Libraries		
Next.js, React, TypeScript	Open-source (MIT License)	0
Tailwind CSS, Radix UI	Open-source (MIT License)	0
Firebase SDK	Free (Google)	0
Google Genkit AI	Free tier available	0
Lucide React Icons	Open-source (ISC License)	0
Recharts Library	Open-source (MIT License)	0
Miscellaneous		
Report Printing	5 copies, hard binding	2,500
Pen Drive	Project softcopy & PPT	300
Total Estimated Cost		**9,600–12,300**

Resource Allocation

The following table shows the resource allocation across team members:

Table 9.3: Resource Allocation

Team Member	Primary Role	Responsibilities
Rahul Tukaram Shendkar	Frontend Developer	Dashboard UIs, responsive design, Tailwind CSS styling
Tushar Tulashidas Pawar	Backend Developer	Firebase services, API routes, Razorpay integration
Karan Naganath Bandgar	Full-Stack Developer	Authentication, location services, notifications
Saurabh Balasaheb Zendage	Testing & Documentation	Test cases, security rules, project report

Chapter 12:
Applications and Future Enhancements

Real-World Applications

AgriTrace has a wide range of applications across various sectors:

1. **Agricultural Cooperatives:** Farming cooperatives can use AgriTrace to coordinate waste collection across multiple farms, optimizing logistics and reducing per-farm costs. The platform's listing system allows cooperatives to aggregate waste from multiple small farmers into larger, more economically viable collection batches.
2. **Government Agencies:** State pollution control boards can leverage the platform's analytics to monitor waste diversion in real-time and enforce anti-burning regulations. The GPS-based location tracking provides verifiable evidence of waste collection activities, supporting regulatory compliance monitoring.
3. **Recycling Industries:** Biomass power plants, compost manufacturers, and biofuel producers can source raw materials through the marketplace. The category-based listing system allows recyclers to filter waste by type, ensuring they receive materials suitable for their specific processing requirements.
4. **Carbon Credit Markets:** Organizations seeking carbon offsets can use AgriTrace's verified waste diversion data for carbon credit generation. The platform's automatic carbon offset calculation, based on scientifically established emission factors, provides a quantifiable measure of environmental impact.
5. **NGOs and CSR Initiatives:** Non-governmental organizations and corporate social responsibility programs can sponsor waste collection through the platform, tracking the impact of their investments through the analytics dashboard.
6. **Research Institutions:** Agricultural universities and research bodies can analyze waste generation patterns, seasonal trends, and regional variations using the platform's aggregated data.
7. **Insurance Companies:** Agricultural insurance providers can use waste management data as a proxy for farm management quality, potentially offering premium discounts to farmers who actively manage their crop residue.
8. **Smart City Projects:** Municipal authorities implementing smart city initiatives can integrate AgriTrace into their sustainable development programs, particularly in peri-urban areas where agricultural waste management intersects with urban pollution control.

Future Enhancements

The following enhancements are planned for future versions of AgriTrace:

1. **Multi-Language Support:** Add Hindi, Marathi, Punjabi, and other regional language interfaces using next-intl for internationalization. This is critical for adoption among farmers with limited English proficiency.
2. **Mobile Application:** Develop native Android and iOS applications using React Native or Flutter for better mobile experience, push notifications, and offline-first functionality.
3. **IoT Integration:** Integrate IoT sensors for automated waste weight measurement and quality assessment at collection points. This would include Bluetooth-connected weighing scales and moisture sensors.
4. **Route Optimization:** Implement AI-driven route optimization for collection agents using Google Maps API or OSRM (Open Source Routing Machine) to minimize transportation costs and carbon footprint.
5. **Blockchain Integration:** Use blockchain for immutable waste tracking and verified carbon credit certificates. Smart contracts could automate payment release upon delivery confirmation.
6. **Advanced AI Features:** Enhance AI capabilities for waste image classification using Google Vision API, crop residue yield prediction, and demand forecasting for recycling facilities.
7. **Government API Integration:** Integrate with government databases (Aadhaar, PM-KISAN) for farmer verification and direct benefit transfer for waste diversion incentives.
8. **Real-Time Chat:** Add in-app messaging between farmers and agents using Firebase Realtime Database for coordination of pickup schedules and special instructions.
9. **Subscription Plans:** Introduce premium features for large-scale operations with recurring collection schedules, priority agent assignment, and bulk pricing.
10. **Environmental Dashboard:** Create a public-facing dashboard showing aggregate environmental impact metrics, including total waste diverted, carbon offsets generated, and air quality improvement estimates.

11. **Predictive Analytics:** Use historical data to predict peak waste generation periods, enabling proactive resource allocation for collection agents and recycling facilities.
12. **QR Code Integration:** Generate QR codes for waste bags that can be scanned at each stage of the supply chain, providing tamper-proof tracking from farm to recycling facility.

Chapter 13:
Conclusion

AgriTrace successfully demonstrates how modern web technologies can be leveraged to address one of India's most pressing environmental challenges—agricultural waste management. The platform provides a comprehensive, end-to-end solution that connects farmers, waste collection agents, and recycling administrators through an intuitive, role-based digital ecosystem.

Key Achievements

The following are the key achievements of this project:

1. **Functional Platform:** A fully operational web application with distinct dashboards for Farmers, Agents, and Administrators, each tailored to their specific workflow needs.
2. **Complete Waste Lifecycle Tracking:** End-to-end tracking from waste reporting through collection, transit, delivery, and recycling, with status updates and notifications at each stage.
3. **Environmental Quantification:** Implementation of a carbon credit system with scientifically-backed emission factors that quantifies the environmental benefit of waste diversion, supporting values from 980 to 1,400 kg CO_2 per metric tonne of waste.
4. **Integrated Payments:** Seamless Razorpay payment integration ensuring transparent and secure financial transactions between stakeholders.
5. **Modern Architecture:** A scalable, maintainable codebase built with Next.js 15.5, TypeScript, Firebase, and Tailwind CSS, following industry best practices for web application development.
6. **Security:** Comprehensive Firestore security rules enforcing role-based access control, with deletion prevention on critical collections and owner-based authorization.
7. **Comprehensive Testing:** All 30 test cases passed successfully across functional, security, performance, and responsive testing categories.
8. **AI Integration:** Google Genkit AI integration for intelligent waste classification, demonstrating the potential for AI-assisted agricultural services.

Lessons Learned

The development of AgriTrace provided valuable insights:

1. **TypeScript's Value:** Adopting TypeScript from the start significantly reduced debugging time. Strict type checking caught 23 potential runtime errors during compilation. The investment in type definitions paid dividends in code maintainability and team collaboration.
2. **Firebase's Strengths and Limitations:** Firebase's real-time synchronization and authentication services accelerated development considerably. However, complex aggregation queries required client-side processing, highlighting the trade-offs of NoSQL databases for analytics-heavy applications.
3. **Component-Based Architecture:** The modular, component-based approach using React and Radix UI allowed parallel development across team members with minimal merge conflicts. Reusable components (e.g., StatCard, LocationPicker, PhotoUploader) significantly reduced code duplication.
4. **Agile Methodology:** The sprint-based approach allowed for iterative refinement based on testing feedback. Features that were initially planned (e.g., IoT integration) were deferred to future versions based on priority reassessment.
5. **Security-First Design:** Writing Firestore security rules early in the development process ensured that security was not an afterthought. Testing rules with the Firebase Emulator Suite was essential for validating access control logic.

Social Impact

AgriTrace has the potential to create significant social and environmental impact:

1. **Air Quality Improvement:** By diverting agricultural waste from open burning, the platform directly contributes to reducing air pollution in affected regions, potentially improving air quality for millions of people.
2. **Farmer Income Generation:** Farmers earn revenue from waste that would otherwise be burned, providing an additional income stream. The platform's transparent pricing ensures fair compensation.
3. **Employment Creation:** The collection agent role creates employment opportunities in rural areas, particularly for youth who can leverage smartphone-based platforms for logistics work.

4. **Sustainable Development Goals:** AgriTrace contributes to multiple UN Sustainable Development Goals, including SDG 13 (Climate Action), SDG 12 (Responsible Consumption and Production), SDG 3 (Good Health and Well-being), and SDG 8 (Decent Work and Economic Growth).

In conclusion, AgriTrace demonstrates the viability of technology-driven agricultural waste management. With continued development and wider adoption, the platform has the potential to significantly reduce crop residue burning, improve air quality, generate economic value from agricultural waste, and contribute meaningfully to India's sustainability goals.

Chapter 14:
Bibliography

[1] N. Jain, A. Bhatia, and H. Pathak, "*Emission of Air Pollutants from Crop Residue Burning in India*," *Aerosol and Air Quality Research*, Vol. 14, 2014, PP 422–430.

[2] S. Pandey and V. K. Agrawal, "*Carbon Footprint Assessment of Agricultural Waste Burning in India*," *Environmental Science and Pollution Research*, Vol. 21, 2014, PP 8820–8830.

[3] L. Moroney, "*The Definitive Guide to Firebase*," *Apress, 1st Edition, 2017*.

[4] "Next.js Documentation," *https://nextjs.org/docs*

[5] "Firebase Documentation – Firestore Security Rules," *https://firebase.google.com/docs/firestore/security started*

[6] "TypeScript Official Documentation," *https://www.typescriptlang.org/docs/*

[7] "Tailwind CSS Documentation," *https://tailwindcss.com/docs*

[8] "Radix UI Primitives," *https://www.radix-ui.com/primitives*

[9] "Razorpay Integration Guide," *https://razorpay.com/docs/*

[10] "Google Genkit AI Framework," *https://firebase.google.com/docs/genkit*

[11] Ministry of Agriculture and Farmers Welfare, "*National Policy for Management of Crop Residues*," *Government of India, 2014*.

[12] Indian Agricultural Research Institute (IARI), "*Crop Residue Management: Challenges and Solutions*," *IARI Publication, New Delhi, 2019*.

[13] "Recharts – A Composable Charting Library for React," *https://recharts.org/*

[14] "React Hook Form – Performant Form Validation," *https://react-hook-form.com/*

[15] "Zod – TypeScript-first Schema Validation," *https://zod.dev/*

[16] "Cloudinary Documentation – Image Upload API," *https://cloudinary.com/documentation/image_upload*

[17] "Lucide React – Beautiful & Consistent Icons," *https://lucide.dev/guide/packages/lucide-react*

[18] "OpenStreetMap Nominatim – Geocoding Service," *https://nominatim.org/release-docs/latest/api/Rever*

[19] R. Kumar, S. Sharma, and A. Singh, "*IoT-Based Monitoring of Agricultural Residue Burning*," *IEEE Sensors Journal, Vol. 20, No. 8, 2020, PP 4523–4531*.

[20] A. Patel, M. Desai, and R. Verma, "*Mobile-First Design for Rural Indian Applications*," *International Journal of Human-Computer Studies, Vol. 131, 2019, PP 45–58*.

[21] B. W. Barry, "*Software Engineering Economics*," *Prentice Hall, 1981*.

[22] "Vercel Platform Documentation," *https://vercel.com/docs*

[23] "State of CSS 2023 Survey Results," *https://2023.stateofcss.com/*

[24] "Node.js Best Practices," *https://nodejs.org/en/docs/guides*

[25] Central Pollution Control Board (CPCB), "*Air Quality Monitoring Data – National Air Quality Index*," *Government of India, 2023*.